基于多阶段优化的微网购电与储能调度策略研究

摘要

微网作为集成分布式电源、储能与负荷的小型发用电系统，通过协调储能充放电与外部购电，可有效促进能源就地消纳并缓解并网压力。本文以购电费最小和供电可靠性为核心，围绕计划购电量与储能充放电策略，建立线性规划、两阶段随机优化及多阶段滚动优化模型，并针对不同信息条件给出相应调度方案，为微网能量管理与经济调度提供了理论参考。

针对问题一，在电价、负载和光伏预测均已知的设定下，以全天购电费最小为目标，建立含功率平衡、储能电量递推、始末电量相等及充放电功率约束的线性规划模型，求得全天购电量 59482.70 kWh、购电费 35126.95 元，储能运行呈“谷充峰放”特征。为避免单一线性规划求解的潜在误差，本文引入动态规划模型 和贪婪算法对求解结果进行交叉验证，DP 购电费较 LP 仅高 0.27%，Greedy 高 12.53%，且 LP 与 DP 的 SOC 轨迹和购电曲线基本一致。结果表明 LP 解满足全部约束，数值稳定且接近全局最优。

针对问题二，考虑负载与光伏功率不确定及缺口需按 5 倍电价紧急购电的机制，建立“计划—补偿”两阶段随机优化模型。本文采用前 5 日滑动平均预报净负荷，基于报童模型确定临界比，并取 80% 分位预测误差作为安全裕量；阶段一以计划购电费最小为目标优化购电量与充放电策略，阶段二按实际缺口进行紧急购电。全年总购电费 1792.34 万元（计划 1654.73 万 + 紧急 137.61 万，紧急占比 7.68%）；对临界比敏感性的分析显示总费用在 0.80—0.85 区间变化平缓，$q \approx 0.85$ 处最小，说明 80% 分位安全裕量策略具有稳健性。

针对问题三，本问新增 0:00、6:00、12:00 和 18:00 四个预报时刻，本文建立了含充放电 0-1 互斥约束的混合整数线性规划模型，并采用多阶段滚动优化框架。首先通过线性插值将逐小时光伏预报降尺度为 10 分钟序列，其次以计划购电费、违约费、超额费和紧急购电费之和最小为目标，在 6:00、12:00、18:00 根据最新预报滚动修正购电计划。全年总购电费 1501.08 万元，较“仅依据 0:00 预报”的基线策略节省 47.57 万元，表明多时刻光伏预报信息具有明确的经济价值。

针对问题四，将固定电价拓展为实时波动电价，重新求解问题二和问题三。波动电价增强了日内峰谷套利动机，但也提高了预测偏差与紧急购电的风险成本。本文对比了固定电价两阶段、固定电价滚动、波动电价两阶段和波动电价滚动四种策略，结果显示滚动策略在两种电价机制下均明显优于两阶段策略；波动电价使总费用小幅上升，但策略结构差异仍是主导因素，其中“滚动 + 波动电价”策略在四种策略中保持最低总费用（1561.33 万元），体现出滚动优化对电价不确定性具有较强适应性。

本文从确定性线性规划，逐步引入预测不确定性多阶段滚动调整和实时波动电价，构建了由线性规划、两阶段随机优化、混合整数线性规划与滚动优化组成的递进式调度模型体系。该体系可推广至含多类型分布式电源、多储能单元及需求响应的综合能源系统，为不确定性环境下微网购电与储能协同调度提供可复用的建模框架和决策参考

关键词：微网调度 储能套利 线性规划 报童模型 滚动优化 模型预测控制

1　问题重述

1.1　问题背景

在“双碳”目标与新型电力系统建设背景下，居民小区配置屋顶光伏和储能设备已成为提升可再生能源利用水平的重要方式。然而，光伏发电受昼夜和天气影响明显，其出力高峰往往与小区用电高峰不完全一致：白天光伏可能大量富余，傍晚负荷升高时又供电不足。若余电直接反送电网，可能引起电压波动和反向潮流；若在高电价时段向外网购电，则会增加运行成本。

微网将光伏、储能和小区负荷纳入统一管理，为协调供需、降低购电费用提供了有效手段。本文需基于题目给出的分时电价、小区负荷、光伏发电量和储能当前储电量等条件，确定完整的购电策略，使得在满足负荷需求的同时尽可能节省购电费用。该问题可归结为分时电价下微网的优化调度问题。

1.2　问题提出

问题 1　要求在电价与小区负载恒定的条件下，依据已知的光伏发电功率预测，以微网供电量不低于小区负载和储能设备一天中的始末储电量相同为约束条件，建立全天购电计划的优化模型，并给出指定时段的购电量、充放电量及全天购电费用。

问题 2　在电价恒定而负载与光伏发电功率变化的场景下，允许微网在供电不足时以 5 倍电价紧急购电，需建立计入紧急购电费用的购电策略优化模型，并针对指定日期给出购电与储能调度结果。

问题 3　在负载与光伏发电功率时变、且可获得未来 24 小时整点光伏功率预报的条件下，微网可在 0:00、6:00、12:00、18:00 四个时刻依据最新预报滚动调整购电策略，其中计划购电量缩减的部分按交易时刻电价的 50% 计违约费用、增加的部分按 1.5 倍计超额费用，要求基于附件 3 的光伏预报数据建立购入计划、紧急与调整三类费用的滚动优化模型，给出指定日期的购电与储能调度结果，并分析是否需要引入更多预报时刻。

问题 4　本问新引入了外网电价实时波动的条件，要求针对波动电价建立微网当天购电策略的优化模型，基于附件 4 的波动电价数据，分别在波动电价下重新求解问题 2 与问题 3，并给出相应的购电与储能调度结果。

2　问题分析

2.1　问题一的分析

问题 1 要求在某一天内，给定分时电价、小区负载和光伏发电预测，制定储能充放电与购电计划，使全天购电费最小，并满足储能始末储电量相同的周期约束。该问题本质上是一个单日确定性线性规划问题。核心在于分时电价存在峰谷价差，储能可通过“低谷充电、高峰放电”实现套利，从而降低购电费用；目标函数与约束均为决策变量的线性函数，且储能电量为线性关系，问题为凸的线性规划，根据性质可以求解出全局的

最优解。建模思路为：首先以每个 10 分钟时段为粒度，定义购电功率、充放电功率与储电量四个决策变量；其次建立功率平衡约束与储能电量方程；最后以全天购电费用最小为目标函数进行求解，并输出指定时段的购电量、充放电量及储电量；在线性规划的结果之后使用动态规划（DP）和贪婪思路交叉验证 LP 解的可靠性。

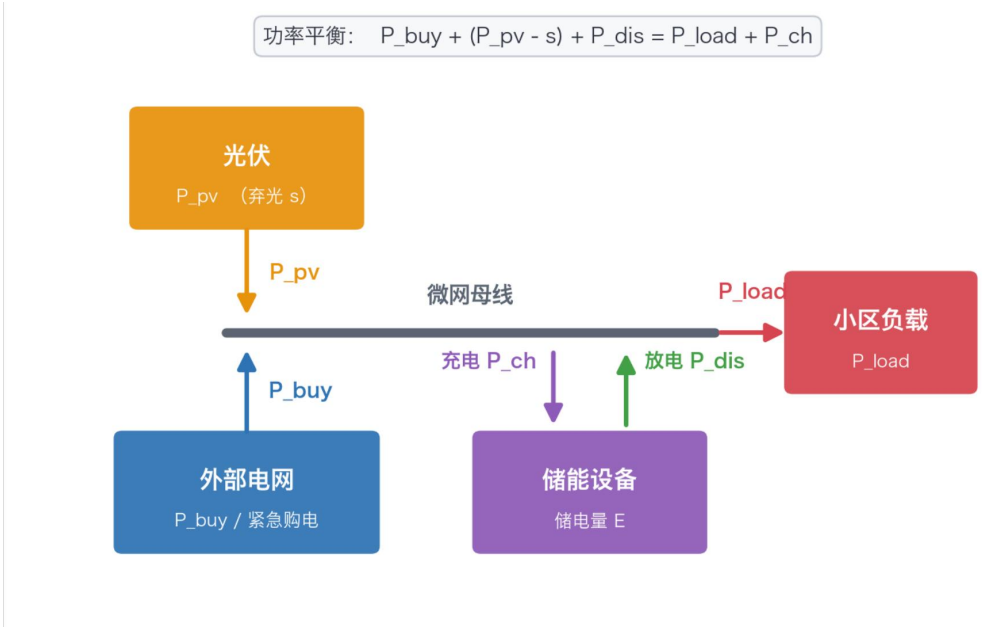


图 1　微网拓扑与能量流示意

2.2　问题二的分析

问题 2 将决策尺度由问题 1 的单日拓展至全年，并引入了预测偏差这一不确定因素。微网在每日 0:00 制定计划时，可依据的仅有负载与光伏的预测值，实际值须待当日方能确定。若按预测制定的计划无法满足实际供电需求，不足部分只能以 5 倍电价的紧急购电弥补，由此形成不对称的惩罚结构：计划超购时，多余电量仅按平价计损；计划欠购时，缺口则须承担 5 倍电价。求解分两个阶段：先利用历史同期数据生成负载与光伏预测，据此建立报童 + 确定性 LP 模型，得到每日计划购电量与储能充放电方案；再以附件 2 的实际值核算计划与实际之间的缺口，缺口即对应紧急购电量，目标函数同时计入计划购电费与紧急购电费。为验证结果的可靠性，并与报童模型的服务水平交叉比对。

2.3　问题三的分析

问题 3 在问题 2 的基础上，多引入了三个时间段对于未来进行滚动预测这一变化因素。每日 0 点更新的相关计划会在后续整点更新时段进行调整，需要将整点 24 点转化为 10 min 144 点从而使得粒度对齐，实际值需待时间到达之后方能确定。问题三对于供电不足和供电超额的两个区域进行了非对称的惩罚结构：供电不足部分购电按照 5

倍价格进行补齐，同时购电超过的部分对于退回需要进行 50% 的罚款操作。因此本题需要在预报更新降低电费和支付调整成本之间进行权衡从而求得最佳优化值；结合只用一次数据进行的预测值的优化对比多次预测的最优化结果，进行算法的可靠性检验。

2.4 问题四的分析

问题 4 将外网电价由固定分时电价推广为全年实时波动的电价，要求在波动电价下重新求解问题 2 与问题 3。该问题本质上是在时变价格参数下对问题 2、3 的推广，与现实中情况十分相似。其核心难点在于目标函数中的价格系数随时间波动，且需明确电价在决策时刻是否可知——若假设在决策时刻已知电价，则退化为含时变价格系数的确定性优化；若电价需预测，则需引入电价不确定性。建模思路为：保持问题 2、问题 3 的模型结构不变，将目标函数中的电价系数替换为附件 4 的波动电价，分别重新求解得到波动电价下的计划、紧急与调整购电策略。

3 模型假设

3.1 总体模型假设

1. 忽略电线传输过程中的电量损耗，总体满足“负载 + 充电 = 购电量 + 放电量 + 光伏发电”。
2. 忽略储能自放电，充放电效率均取 90%（$\eta_c = \eta_d = 0.9$），充放电不同时进行。
3. 仅考虑购电相关费用，包括计划购电费、紧急购电费和调整偏差费用，忽略设备运维、折旧、寿命损耗等其他成本。

3.2 问题一的模型假设

1. 各十分钟时段内电价、负载、光伏发电功率和充放电功率视为常数。
2. 假设每天 0 时和 24 时的储能电量相同，储电量取六千千瓦时。

3.3 问题二的模型假设

1. 假设通过相关的基准预测数据集的数据可以预测出当天相关数据的大概值。
2. 储能电量跨日连续，前一日二十四时电量作为后一日零时初始电量，不要求每日回到同一水平。

3.4 问题三的模型假设

1. 假设粒度对齐，从整点 24 点转化成为 10 min 144 点间隔进行插值的操作不会大幅度影响相关的数据预测结果。

2. 每一次预报都会对于整体的模型进行优化，优化的结果持续到下一次预测结果产生的时间节点。

3. 假设总购电费 = 计划购电费 + 紧急购电费 + 违约费用。

4. 每日 0 点的相关小区负载和光伏发电功率均视为预测值。

3.5 问题四的模型假设

1. 情景一，决策时所需时段的实时电价视为已知，可通过对应位置的实时电价信息直接获取，无需进行电价预测；情景二，决策时所需时段的实时电价未知，仅能依据历史实时电价变化进行预测，根据预测电价制定计划和调整策略。

2. 紧急购电费用、违约费用和超额费用的参考价格均为交易时刻实际实时电价，而非附件一中给出的固定分时电价。

4 符号说明

表 1 主要符号说明

符号	说明	量纲
λ_t	第 t 时段电价（附件 1 分时 / 附件 4 波动）	元/kWh
L_t	第 t 时段小区负载	kW
P_t	第 t 时段光伏发电功率	kW
N_t	第 t 时段净负荷，$N_t = L_t - P_t$	kW
η	充放电效率	–
$P_{\max}$	最大充放电功率	kW
$E_{\max}/E_{\min}$	储能最大/最小容量	kWh
$E_{\text{upper}}/E_{\text{lower}}$	储能运行上/下限	kWh
E_0	初始储电量	kWh
b_t	第 t 时段向外部电网购电功率	kW
c_t	第 t 时段储能充电功率	kW
d_t	第 t 时段储能放电功率	kW
s_t	第 t 时段弃光功率	kW
E_t	第 t 时段末储电量	kWh
e_t	第 t 时段紧急购电功率（实时缺口补偿）	kW
C	总购电费	元

5　问题一的模型建立与求解

5.1　模型的选择和准备

5.1.1　线性规划模型

在本模型的设定中，光伏发电功率、小区负载和电价在制定计划时均被设为确定性参数，在此基础上构建的目标函数及后续约束条件均满足齐次线性条件。鉴于上述数学特性，采用凸线性规划模型求解能满足线性规划问题对于最优化的计算，同时在目标函数符合凸的性质的条件下可保证在有约束的情况下能优化到唯一的最优值。

5.1.2　动态规划模型（DP）

由于凸线性规划模型仅能给出数值意义上的最优解，结果正确性缺乏独立的证据支撑，难以直接确认其可靠性。本文引入动态规划（DP）对线性规划的求解结果进行算法层面的可靠性检验。动态规划是依据最优子结构性质与贝尔曼最优性原理，将整体问题分解为若干相关的子问题，利用子问题间的状态转移方程，自底而上地递推对各子问题逐级判断与递归寻优，综合子问题的最优解确定全局最优解。

5.1.3　贪婪算法

为检验线性规划模型结果的合理性，构造一种“谷时段充电、峰时段放电、日末电量校正”的启发式调度规则，生成满足功率平衡、储能容量、充放电功率及日末电量约束的可行调度方案。该方案作为独立于线性规划的参考解，将启发式调度费用与线性规划最优费用、动态规划求解结果进行交叉验证。

5.2　数据分析和预处理

5.2.1　数据来源说明

问题一的数据仅涉及附件 1：某日分时电价、小区负载 L_t 和光伏发电预测功率 P_t（$t = 1, 2, \ldots, 144$，一天 144 个点分段）。计划时相关的实际光伏发电数据尚无法获取，附件 1 中是光伏的预测功率，问题一将其视为已知量进行优化计算。

5.2.2　分时电价、负载和光伏的统计特征

分时电价曲线如图 2 所示。电价范围 0.3713 ∼ 1.3952 元/kWh，全天均价 0.7662 元/kWh，峰谷价差达 1.0239 元/kWh（约 2.76 倍）。低谷集中于凌晨 0:00—6:00（约 0.43 元/kWh）与夜间 22:00—24:00（约 0.42 元/kWh）；高峰集中于傍晚 18:00—20:00（约 1.31 元/kWh），于 20:30 达全天峰值 1.3952 元/kWh；上午 8:00—10:00 出现次高峰（约 1.03 元/kWh）；中午 10:00—14:00 电价相对回落（约 0.67 ∼ 0.85 元/kWh），恰与光伏大发时段重合。

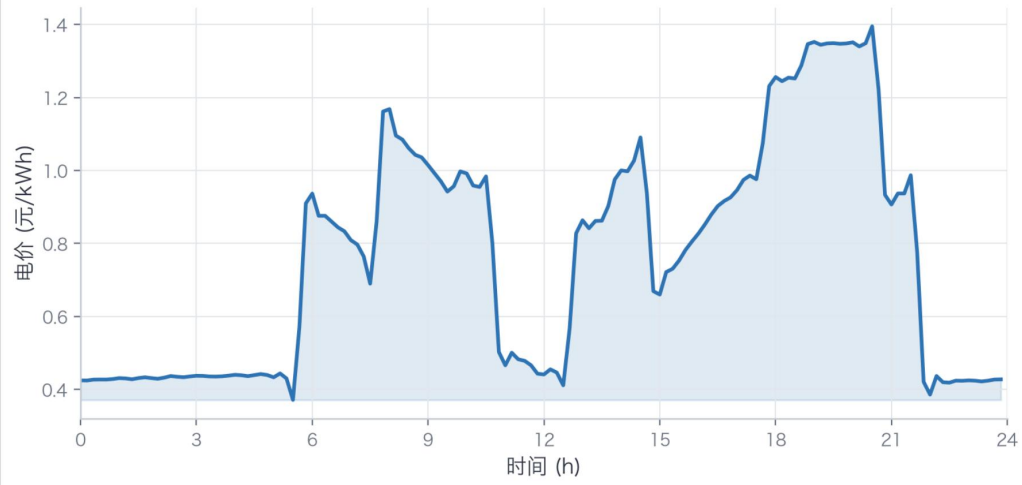


图 2　分时电价曲线

负载与光伏预测功率曲线如图 3 所示。光伏大发（中午）与负载高峰（早、晚）并不同步——光伏富余集中在中午，而傍晚电价最高、负载仍较高时，光伏已接近零出力。这一错位正是储能发挥“削峰填谷、时移能量”作用的物理前提。

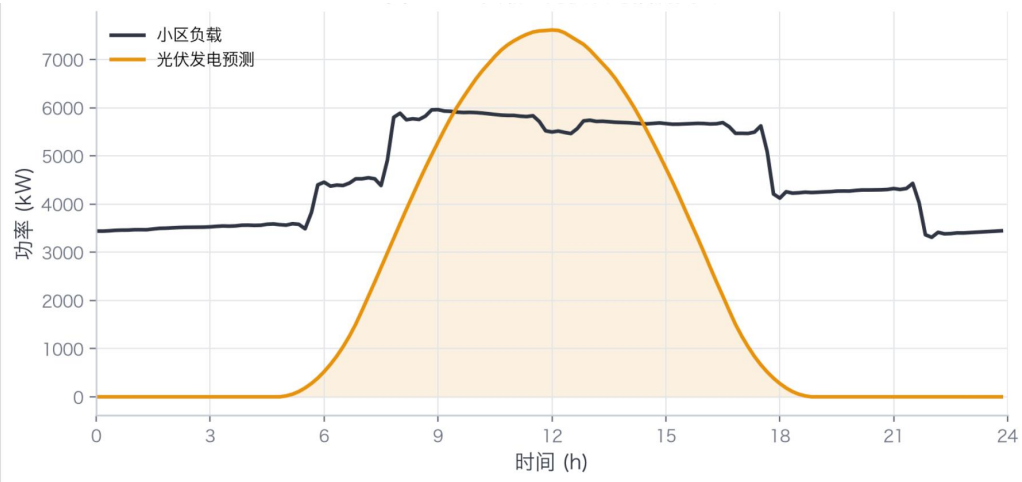


图 3　负载与光伏预测功率曲线

5.2.3　模型建立

调度的经济目标是在保证负载供电的前提下，使全天向电网购电的费用最小，目标函数记为

$$\min C = \sum_{t=1}^{144} p_t \, b_t \, \Delta t \tag{1}$$

其中 p_t 为附件 1 给出的分时电价，$\Delta t = 1/6$ h 为时段步长。约束条件如下。

（1）功率平衡约束：

$$b_t - s_t + d_t - c_t = L_t - P_t \tag{2}$$

（2）储能电量递推约束：

$$E_t = E_{t-1} + \eta\, c_t\, \Delta t - \frac{d_t\, \Delta t}{\eta} \tag{3}$$

（3）始末电量约束：

$$E_{144} = E_0 = 6000 \text{ kWh} \tag{4}$$

（4）变量范围约束：

$$0 \le c_t, d_t \le P_{\max}, \qquad 0 \le s_t \le P_t, \qquad 1200 \le E_t \le 10800 \tag{5}$$

至此，目标函数式 (1) 与约束式 (2)—(5) 均为决策变量的线性函数，构成一个标准的线性规划模型。其可行域为凸集，故符合凸函数局部最优即全局最优的性质，可直接由成熟求解器求得唯一最优解，所得结果将在 5.4 节由动态规划与贪婪算法进一步交叉验证。

5.3　模型求解和结果分析

表 2 给出微网在若干代表时刻的购电量以及全天汇总结果。表中购电量的分布呈现出清晰的“避峰购电”特征。10:00、14:00、20:00 三个时刻购电量均为零：10:00 处于上午次高峰电价区间，20:00 则逼近全天电价峰值，这两个时段光伏出力或储能放电已足以覆盖负载，模型选择“一分不买”；14:00 光伏出力仍强，负载可由光伏直接满足。16:00 与 18:00 两个时刻购电量较高——此时光伏明显衰减而负荷仍处高位，储能虽在全力放电仍留缺口，只能向外部购电。值得注意的是 12:00 的 480.41 kWh：此时虽然光伏发电功率较大、负载使用不存在缺口，但此时的用电费用较低，模型选择在此低价时段购电。

表 2　微网在指定时间段的购电量及全天购电量/购电费

时间段	购电量 (kWh)
10:00—10:10	0.0000
12:00—12:10	480.4124
14:00—14:10	0.0000
16:00—16:10	445.4317
18:00—18:10	531.8940
20:00—20:10	0.0000
全天购电量	59482.6990
全天购电费	35126.9486 元

表 3 按 4 小时块聚合储能的充放电量，并给出始末储电量。充放电的时序与电价、光伏双曲线图的趋势高度吻合，形成典型的“峰放谷充”运行格局：凌晨 0:00—4:00 为全

天电价最低区间，储能以最大功率充电 4500 kWh（谷充）；4:00—8:00 电价抬升、早高峰负荷来临，储能放电 6365.84 kWh 分担负荷；8:00—12:00 为上午次高峰，储能继续放电 1702.99 kWh，同时于午间电价回落处开始回充 4787.96 kWh；12:00—16:00 光伏大发，储能大量充电 5286.04 kWh，把午间富余光伏几乎全部吸收；16:00—20:00 光伏骤减、晚高峰电价最高，储能全力放电 5780.13 kWh，为全天最大放电时段；20:00—24:00 夜间电价再次走低，储能回充 5333.33 kWh，为次日循环蓄势。始末储电量均为 6000 kWh，满足“日初日末等电量”约束，保证模型可按天独立重复运行。

表 3　储能设备在指定时间段的充放电量及储电量

时间段	充电量 (kWh)	放电量 (kWh)
0:00—4:00	4500.0000	0.0000
4:00—8:00	833.3333	6365.8412
8:00—12:00	4787.9643	1702.9970
12:00—16:00	5286.0352	91.1014
16:00—20:00	0.0000	5780.1319
20:00—24:00	5333.3333	2859.8681
0:00 储电量	6000.00	–
24:00 储电量	6000.00	–

5.4　模型结果的可靠性验证

5.4.1　验证思路与评价指标

为排除单一线性规划求解器可能存在的数值误差，并检验所得调度策略的物理可实现性，本文以线性规划结果作为基准，分别采用动态规划（DP）和贪婪算法（Greedy）进行交叉验证。选取全天购电费、储能 SOC 轨迹、购电策略曲线以及约束满足情况作为评价指标。若不同算法在费用和调度轨迹上高度一致，则说明线性规划结果可靠；若启发式算法费用明显更高，则进一步说明线性规划在给定模型下实现了更优的经济调度。

5.4.2　结果对比分析

图 4 给出了三种算法的全天购电费对比；储能 SOC 轨迹和购电功率策略的对比分别见附录图 A3 和图 A4。由图可知，线性规划 LP、动态规划 DP 和贪婪算法 Greedy 的全天购电费分别为 35126.95 元、35222.69 元和 39526.37 元。以 LP 为基准，DP 费用仅高 95.74 元，相对偏差约 0.27%；Greedy 费用高 4399.42 元，相对偏差约 12.53%。LP 费用最低，DP 非常接近，说明 LP 结果在数值上稳定且接近全局最优；Greedy 因规则固定，经济性明显较差。

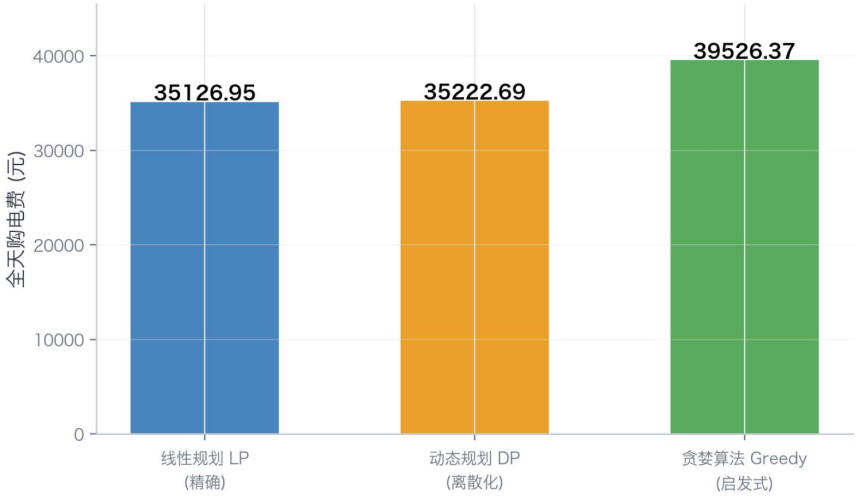


图 4　三种算法购电费对比（交叉验证）

进一步结合附录图 A3 和图 A4 可知，LP 与 DP 的储能 SOC 轨迹几乎重合，购电功率曲线也基本一致，二者在谷时段充电、早高峰放电、午间光伏富余充电、晚高峰放电、夜间回充等关键时段的调度时序相同；Greedy 轨迹在部分时段偏离明显。上述结果表明，LP 与 DP 在费用和调度策略两个层面相互印证，LP 结果满足功率平衡、储能容量、充放电功率及始末电量约束，具有较高的数值可靠性和物理可实现性。

综上，线性规划结果通过了动态规划与贪婪算法的交叉验证，可作为问题一的最终计划购电策略。DP 与 LP 的微小费用差异主要来源于状态和动作离散化，Greedy 费用偏高则源于其规则固定、缺乏全局寻优能力。

6　问题二的模型建立与求解

6.1　模型的选择和准备

6.1.1　滑动平均预报模型

滑动平均预报模型用于结合近期的平均水平来代表未来的趋势，消除随机波动的影响。问题二中，采用该模型对历史负载和光伏序列进行短期预测，以预测均值作为净需求的期望估计，结合报童临界比确定各时段最优计划购电量，在紧急购电惩罚下最小化期望购电费用。

6.1.2　报童临界比敏感性

问题二采用报童模型，最优购电量由临界比决定。临界比依赖于紧急购电电价倍数、储能残值、充放电效率等参数，这些参数当天不可知。因此，必须通过临界比敏感性分析，检验购电策略对参数变化的稳健性，并为实际决策提供依据。

6.2　数据分析和预处理

问题二中的数据来自于附件 1 中的电价和附件 2 中的数据，实际的负载和光伏发电数据来自相关的滑动平均预报模型给出的预测数据。

6.3　模型建立

问题二引入源荷不确定性：每天的电价仍由附件 1 给出且当日各时段电价已知，但小区负载与光伏发电功率随时间变化，且实际值只有当时是未知的，因此，0:00 制定计划时只能依据历史数据形成预报，并需为预报偏差预留安全裕量。本文采用前 5 日实际值的滑动平均作为当日预报，即

$$\hat{L}_d(t) = \frac{1}{\min(5,d)} \sum_{k=1}^{\min(5,d)} L_{d-k}(t), \qquad \hat{P}_d(t) = \frac{1}{\min(5,d)} \sum_{k=1}^{\min(5,d)} P_{d-k}(t) \tag{6}$$

其中不足 5 日时用已有实际值平均，第 1 日以附件 1 的单日数据作为先验。定义净负荷

$$N_d(t) = L_d(t) - P_d(t), \qquad \hat{N}_d(t) = \hat{L}_d(t) - \hat{P}_d(t) \tag{7}$$

其预报误差

$$e_d(t) = N_d(t) - \hat{N}_d(t) \tag{8}$$

反映了预测误差。由于微网供电量不得低于小区负载，一旦计划覆盖不足，缺口部分需以交易时刻电价的 5 倍紧急购电，而计划超购部分仅按平价计入成本。设电价为 p_t，则欠购的边际成本和超购的边际成本为

$$c_u = 5p_t - p_t = 4p_t, \qquad c_o = p_t \tag{9}$$

报童模型临界比为

$$q^* = \frac{c_u}{c_u + c_o} = \frac{4p_t}{4p_t + p_t} = \frac{4}{5} = 0.8 \tag{10}$$

据此，取预报误差的逐时段 80% 分位作为安全裕量

$$q_{80}(t) = \hat{F}_t^{-1}(0.8) \tag{11}$$

并以“净负荷预报 + 安全裕量”作为阶段一的确定性输入

$$\hat{N}_d(t) + q_{80}(t) \tag{12}$$

阶段一为刚性计划：在 0:00 一次性确定计划购电量 $b_d(t)$、充放电功率 $ch_d(t), dis_d(t)$ 及储能电量 $E_d(t)$，满足功率平衡、储能容量与功率约束、储能跨日连续约束，目标为计

划购电费用最小：

$$\min \sum_{t=0}^{143} p_t \, b_d(t) \, \Delta t \tag{13}$$

该模型是“计划—补偿”两阶段随机优化：第一阶段用报童安全裕量对冲预测偏差，第二阶段用 5 倍惩罚价格承担实时缺口，从而在计划经济性与供电可靠性之间取得权衡。阶段二按实际缺口进行紧急购电：

$$e_t = \max\left(0,\ N_d(t) - \left[\hat{N}_d(t) + q_{80}(t) + s_t^*\right]\right) \tag{14}$$

总购电费为

$$C_d = \underbrace{\sum_t p_t \, b_t^* \, \Delta t}_{\text{计划购电费}} + \underbrace{\sum_t 5 \, p_t \, e_t \, \Delta t}_{\text{紧急购电费}} \tag{15}$$

6.4　求解及结果分析

按上述模型对 2025 年 2 月 1 日至 12 月 31 日共 334 天逐日求解，得到全年计划购电费约为 16,547,313.48 元，紧急购电费约为 1,376,134.88 元，总购电费约为 17,923,448.37 元（即 1792.34 万元），紧急购电费占比约 7.68%，全年紧急购电量约 352,068.30 kWh。全年计划购电费、紧急购电费及总购电费的趋势图如图 5 所示。

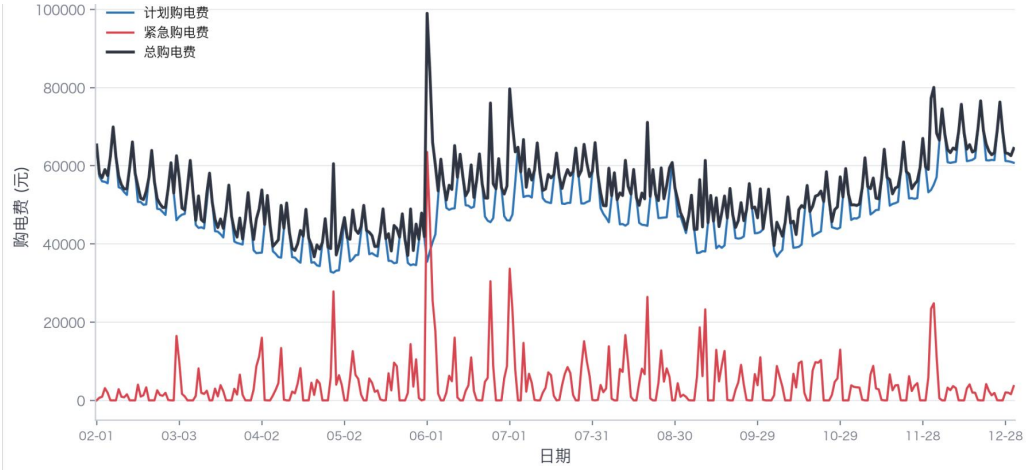


图 5　全年计划购电费、紧急购电费及总购电费

结合图 5，从费用结构上看，计划购电费占大部分（约 92%），说明 80% 分位安全裕量已能覆盖绝大多数净负荷需求；紧急购电费虽占比不高，但在光伏骤降、负载尖峰或预报偏差较大时集中出现，是总成本中不可忽略的惩罚项。

储能方面，result2.xlsx 中充放电量表显示，储能通常在低价或光伏富余时段充电、在净负荷高峰或高价时段放电；储能跨日连续，日末储电量顺延为次日初值，全年末储电量稳定在运行下限 1200 kWh（水价值残值定价下储能当日放空、次日在谷时重新充

电）；全年日均充电量约 22,518.65 kWh，日均放电量约 18,240.10 kWh，充放电效率约 90%，与设备参数一致。

综合来看，采用“前 5 日滑动平均预报 + 80% 分位安全裕量 + 确定性 LP 计划 + 紧急购电补偿”的两阶段策略，能在保证供电不低于负载的前提下有效控制正常电价下的购电成本，并将高倍紧急购电限制在必要范围内。后续可通过提高预报精度、动态调整安全裕量、引入需求响应或滚动优化，进一步降低紧急购电费用。

指定日期（春分、夏至、秋分、冬至）的计划购电量与紧急购电量汇总见表 4，储能充放电量见表 5。其中春分、夏至两日光伏出力充足且预报偏差小，阶段一计划已足以覆盖实际净负荷，故无紧急购电；秋分、冬至两日实际净负荷明显超出预报覆盖，触发实时缺口，需按 5 倍电价紧急购电补偿。

表 4　微网在指定日期的计划购电量与紧急购电量

日期	计划购电量 (kWh)	紧急购电量 (kWh)	总购电量 (kWh)
2025-03-20	77972.3809	0.0000	77972.3809
2025-06-21	87359.4626	0.0000	87359.4626
2025-09-23	67013.0532	2023.4930	69036.5462
2025-12-21	95967.1253	1165.3099	97132.4352
全年合计（334 天）	26339004.78	352068.30	26691073.09

表 5　储能设备在指定日期的充放电量

日期	时间段	充电量 (kWh)	放电量 (kWh)
2025-03-20	0:00–4:00	9000.0000	0.0000
	4:00–8:00	2500.0000	4891.8800
	8:00–12:00	5833.3300	4321.2400
	12:00–16:00	5904.4500	969.4900
	16:00–20:00	0.0000	5519.0800
	20:00–24:00	0.0000	3120.9200
2025-06-21	0:00–4:00	9000.0000	0.0000
	4:00–8:00	2500.0000	4198.4200
	8:00–12:00	5833.3300	4576.5800
	12:00–16:00	6684.0300	2039.0700
	16:00–20:00	0.0000	5833.3300
	20:00–24:00	0.0000	2806.6700
2025-09-23	0:00–4:00	9000.0000	0.0000
	4:00–8:00	1666.6700	6452.1900
	8:00–12:00	5657.0700	2187.8100
	12:00–16:00	5149.5800	113.3800
	16:00–20:00	0.0000	5663.2400
	20:00–24:00	0.0000	2976.7600
2025-12-21	0:00–4:00	9000.0000	0.0000
	4:00–8:00	2500.0000	1508.3300
	8:00–12:00	5833.3300	7806.6700
	12:00–16:00	7964.9900	2536.6400
	16:00–20:00	0.0000	5693.2800
	20:00–24:00	0.0000	2946.7200

6.5　模型结果的可靠性验证

图 6 给出了不同报童临界比下的全年总购电费。由图可知，通过设置不同的报童临界比数值，得出全年总购电费在服务率 $q \approx 0.85$ 处取得数值最小值，与报童模型理论临界比 $q = 0.80$ 仅相差约 0.03%，且总费用曲线在 0.80—0.85 区间极为平坦、变化微小，验证了问题二采用 80% 分位安全裕量的报童型计划策略具有良好的最优性与可靠性。临界比敏感性扫描结果见表 6。

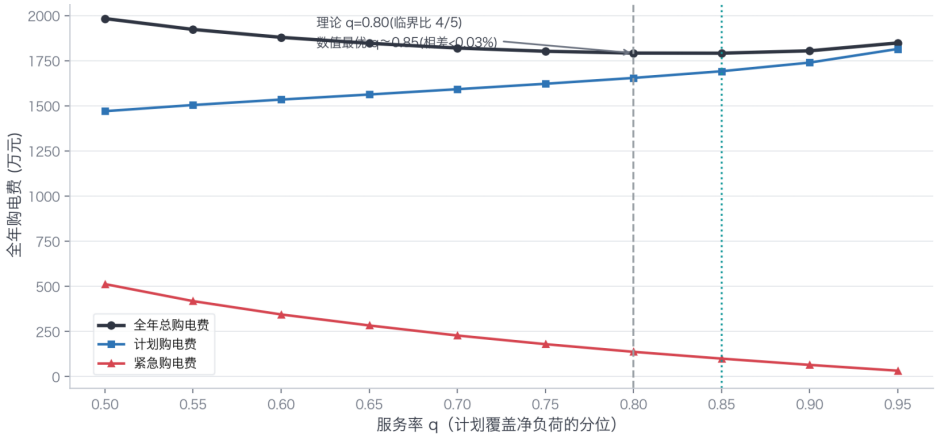


图 6　报童临界比敏感性（全年总购电费随服务率变化）

表 6　服务率敏感性扫描结果

服务率 q	计划购电费 (万)	紧急购电费 (万)	总购电费 (万)
0.70	1592.30	228.03	1820.34
0.80	1654.73	137.61	1792.34
0.85	1691.79	99.96	1791.75
0.90	1739.66	65.19	1804.86

7　问题三的模型建立与求解

7.1　模型的选择和准备

7.1.1　混合整数线性规划模型（MILP）

问题三中 MILP 模型通过在储能的充放电行为之间引入 0-1 互斥变量，保证同时段不能同时充放电的物理约束，并在满足功率平衡、储能容量上下限、充放电功率上限及首末储电量相等等约束的前提下，以购电费用最小为目标求解出每个时段最优的购电量与充放电策略。

7.1.2　多阶段滚动优化模型（MPC）

问题三以 0:00 计划购电量为第一阶段决策，以 6:00、12:00、18:00 根据新光伏预报调整购电量为后续阶段决策，在计划与调整的偏差惩罚（0.5 倍违约、1.5 倍超购）和紧急购电惩罚（5 倍电价）下，建立多阶段滚动优化模型，逐阶段滚动更新购电策略以最小化总购电费用。图 7 为 MPC 原理图。

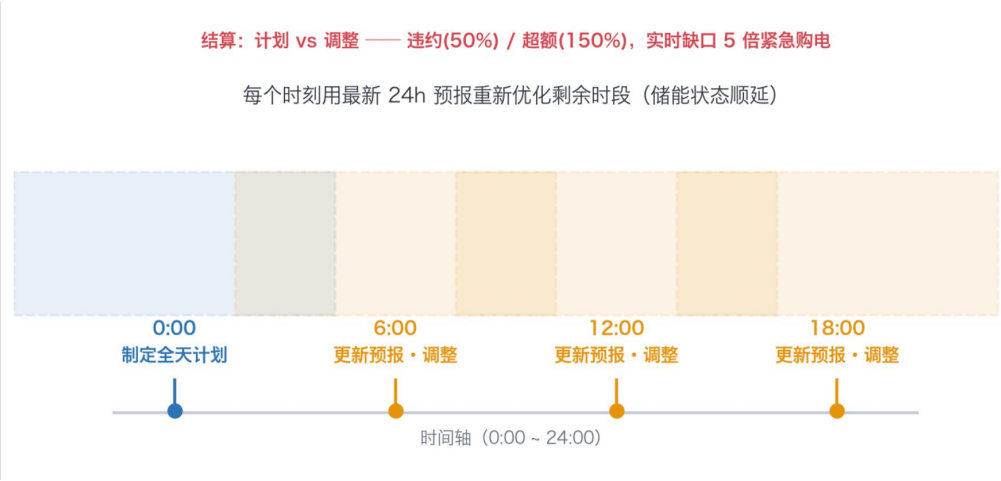


图 7　MPC 滚动优化原理示意

7.1.3　光伏预报时间尺度转换模型（线性插值）

由于附件 3 提供的光伏预报为逐小时整点数据，而微网优化决策的时间分辨率为 10 分钟，因此需通过线性插值将 24 个整点预报值降尺度扩展到 144 个 10 分钟时段的预报序列，并保证相邻时段（如 6:00、12:00、18:00）衔接处预报值的连续性，从而为 MILP 与 MPC 模型提供与决策步长一致、时序连续的光伏功率输入。

7.1.4　三阶段偏差结算模型（违约/超额/紧急）

在问题三中，该模型通过将滚动调整后的实际购电量与日前计划购电量逐时段比较，把购电费用分解为计划内部分、违约部分、超额部分以及实时缺口触发的紧急购电部分，使目标函数能够真实反映微网在不确定光伏预报下的综合购电成本。

7.1.5　对比基线模型（0 点-only）

在问题三中，该模型以仅依据 0:00 预报制定全天计划、不做滚动调整的策略作为对照方案，将其总购电费用与滚动 MPC 策略的总购电费用相减，从而定量评估“6:00、12:00、18:00 获取更新预报并滚动调整”这一信息更新机制为微网带来的经济节省，用以量化光伏预报滚动更新的价值。

7.2　模型建立

问题 3 在问题 2 的基础上进一步引入了多时刻光伏预测信息与购电计划调整机制。每天 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点的光伏发电功率预报，因此微网不再只是在 0:00 一次性制定全天购电计划，而是可以在后续预报时刻根据更新的光伏预测信息对原计划购电量进行调整。设全天划分为 $T = 144$ 个 10 分钟时段，$t = 1, 2, \ldots, T$。令 L_t 表示 t 时段小区负载，P_t 表示光伏实际发电功率；b_t^0 表示 0:00 制定的计划购电量，b_t 表示后续时刻调整后的实际计划购电量，e_t 表示因供电不足而产生

的紧急购电量；E_t 表示储能设备在 t 时段末的储电量，c_t, d_t 分别表示储能充电量和放电量。储能设备最大容量为 12000 kWh，最大充放电功率为 5000 kW，储电量需保持在 $1200 \sim 10800$ kWh 之间，充放电效率均为 90%。以全天总购电费用最小为目标，总费用包括计划购电费用、调整相关费用和紧急购电费用：

$$\min C = C_{\text{plan}} + C_{\text{adj}} + C_{\text{emer}} \tag{16}$$

其中，计划购电费用按 0:00 制定的计划购电量与对应时段电价计算；若后续调整购电量低于原计划购电量，则减少部分视为违约，按交易时刻电价的 50% 计费；若调整购电量高于原计划购电量，则超出部分按交易时刻电价的 1.5 倍计费；若微网提供的电能仍低于小区负载，则不足部分需紧急购电，紧急购电电价按交易时刻电价的 5 倍计算。上述三类费用可写为

$$C_{\text{plan}} = \sum_t \lambda_t b_t^0 \, \Delta t, \tag{17}$$

$$C_{\text{adj}} = \sum_t \Big(0.5\lambda_t \max(b_t^0 - b_t,\, 0) + 1.5\lambda_t \max(b_t - b_t^0,\, 0) \Big) \Delta t, \tag{18}$$

$$C_{\text{emer}} = \sum_t 5\lambda_t \, e_t \, \Delta t \tag{19}$$

约束条件包括功率平衡约束

$$b_t + (P_t - s_t) + d_t - c_t = L_t, \qquad \forall t \tag{20}$$

储能电量递推与容量约束

$$E_t = E_{t-1} + \eta c_t \Delta t - \frac{d_t \Delta t}{\eta}, \qquad E_{\text{lower}} \le E_t \le E_{\text{upper}} \tag{21}$$

充放电互斥与功率约束

$$0 \le c_t \le z_t P_{\max}, \quad 0 \le d_t \le (1 - z_t) P_{\max}, \quad z_t \in \{0, 1\} \tag{22}$$

以及购电量非负、紧急购电量仅在供电不足时产生等约束。由此建立的问题 3 模型是一个以全天购电费用最小为目标的混合整数线性规划模型。通过该模型，可以在 0:00 制定初始计划购电策略，并在 6:00、12:00、18:00 根据最新光伏预报对购电计划进行滚动修正，从而兼顾经济性与供电可靠性。

7.3　求解及结果分析

求解时以附件 1 中的电价为价格输入，以附件 2 中的小区负载和光伏实际功率为运行校验数据，以附件 3 中的光伏发电预报数据作为 0:00、6:00、12:00、18:00 制定和调整购电策略的依据。计算得到各指定日期的计划购电量、调整购电量、紧急购电量、储能

充放电量及储电量变化，并进一步汇总得到全天购电量、计划购电费、调整相关费、紧急购电费和总购电费。从结果看，模型能够根据电价峰谷变化和光伏出力预测灵活调整购电计划：在夜间和清晨光伏出力为零或较低时，微网主要依靠外部电网购电并向储能充电；在中午光伏出力较高时，适当减少外网购电甚至向储能充电，以降低高价时段购电需求；在傍晚光伏出力下降且负载仍较高时，储能放电补充供电，减少紧急购电。调整机制使得 6:00、12:00、18:00 后的购电量能够随最新光伏预报修正，如图 8 所示。

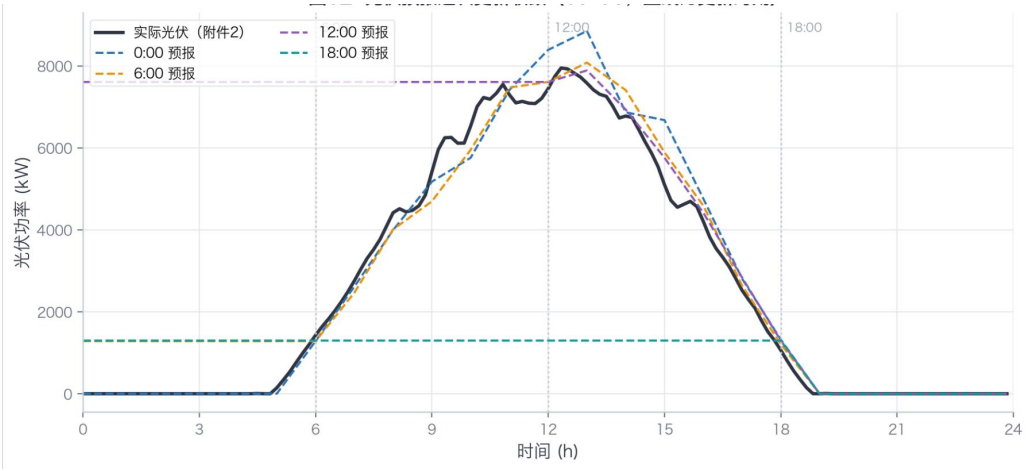


图 8　滚动预报更新对比

原计划购电量高于调整购电量的部分产生违约费用，调整购电量高于原计划购电量的部分产生加价费用，但总体上通过滚动调整降低了紧急购电量和总费用。全年 334 天的费用汇总为：计划购电费 1187.78 万元、调整相关费 67.86 万元、紧急购电费 245.44 万元，总购电费 1501.08 万元（紧急占比 16.35%、调整占比 4.52%）；而仅依据 0:00 预报的基线策略总费用为 1548.66 万元，滚动调整节省 47.57 万元（3.07%）。结果文件中给出了完整的计划购电策略、储能充放电策略和紧急购电结果，说明所建模型能够较好反映多时间尺度预测信息下的微网购电决策过程，并兼顾储能安全运行、供电可靠性和经济性要求。

7.4　是否引入其他时刻预报

经过研究，本文认为不需要引入其他时刻预报，理由如下：其一，所提供的附件 3 中预报时刻仅有 4 个，没有其他预报数据源，刻意引入其他时刻的预报需要额外制定预报更新规律和精度，很有可能偏离题目设定；其二，边际收益递减——对比基线与滚动 MPC 之间的差反映了 1 个决策点到 4 个决策点后带来的节省，若时刻点继续增加，节省的增幅呈递减趋势、接近饱和，反而会导致运算成本增高，不划算。预报更新频率的敏感性验证见附录图 A6。

8　问题四的模型建立与求解

8.1　模型确立和求解思路

结合波动电价的相关条件，同时结合问题二和问题三中的双阶段线性规划问题和滚动规划模型（考虑粒度对齐）以及三阶段偏差结算模型对本问题进行对比的优化求解操作，同时结合问题二和三的无波动电价进行对比，得出相关的最优模型。

8.2　数据分析和预处理

8.2.1　数据来源

问题四的相关数据来源于附件 2 中的小区负载，附件 3 光伏预报和附件 4 中波动电价价格（实际值）。

8.2.2　时变电价数据

固定电价（典型日）和波动电价在 24 小时之内的对比折线图如图 9 所示。

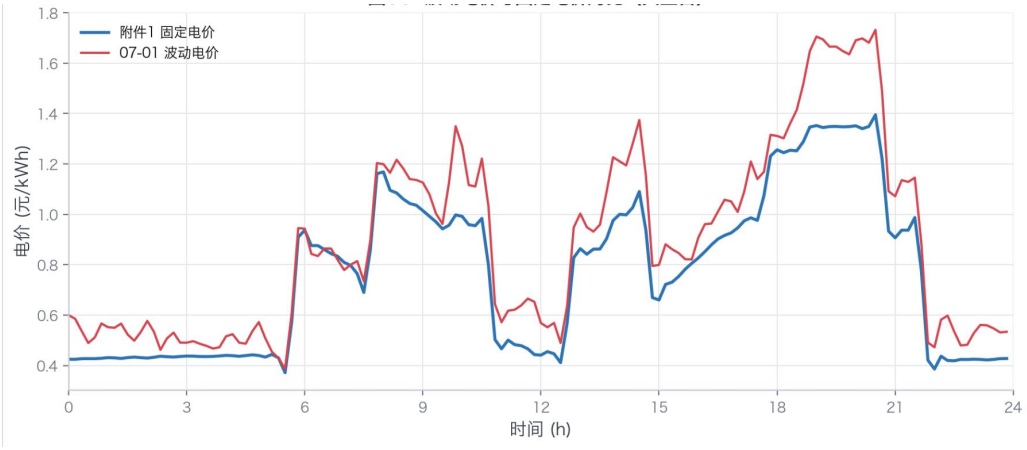


图 9　固定电价（典型日）和波动电价在 24 小时之内的对比折线

时变电价随时间变化（以月为单位）的热力图如图 10 所示。

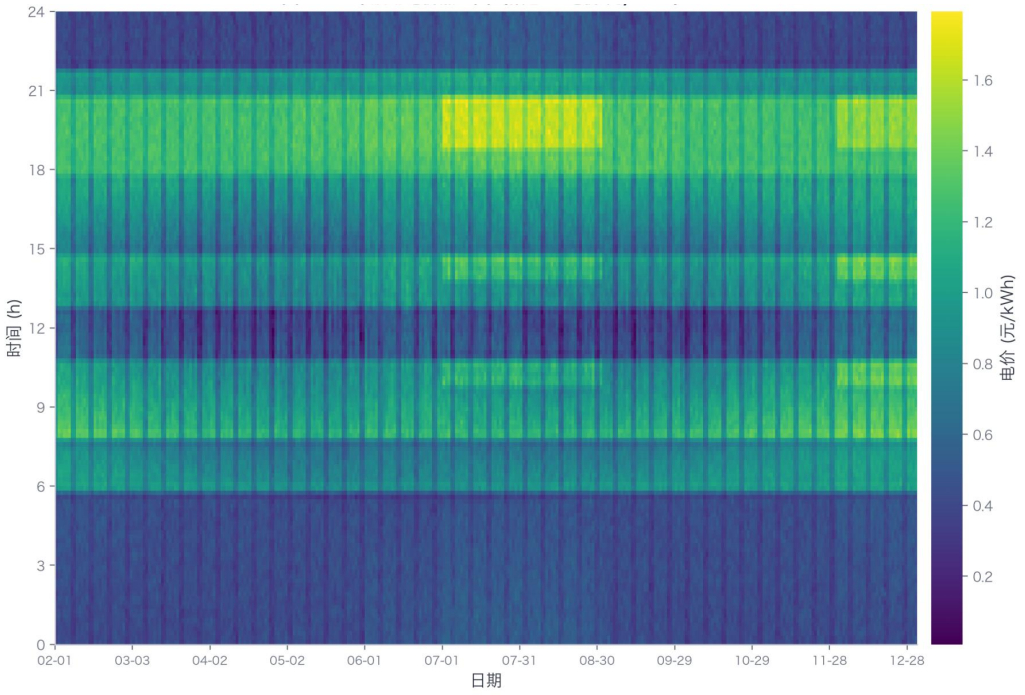


图 10　时变电价随时间变化（以月为单位）的热力图

由上述统计可得两点结论：其一，波动电价的日内峰谷结构与附件 1 一致，储能“谷充峰放”的经济动机依旧成立；其二，波动电价在逐日维度上增加了大量不确定性（日内标准差 0.3407 元/kWh、单日最大偏离 0.5046 元/kWh），这使“固定一日的确定性调度”失效，必须按当日实际波动电价逐日重解，且实时缺口的紧急补偿（5 倍价）在波动电价下成本更高、更需规避。

8.3　求解及结果分析

对于相关算法优化结果和 4 个方法的对比图如图 11 所示。全年 334 天的费用汇总见表 7。

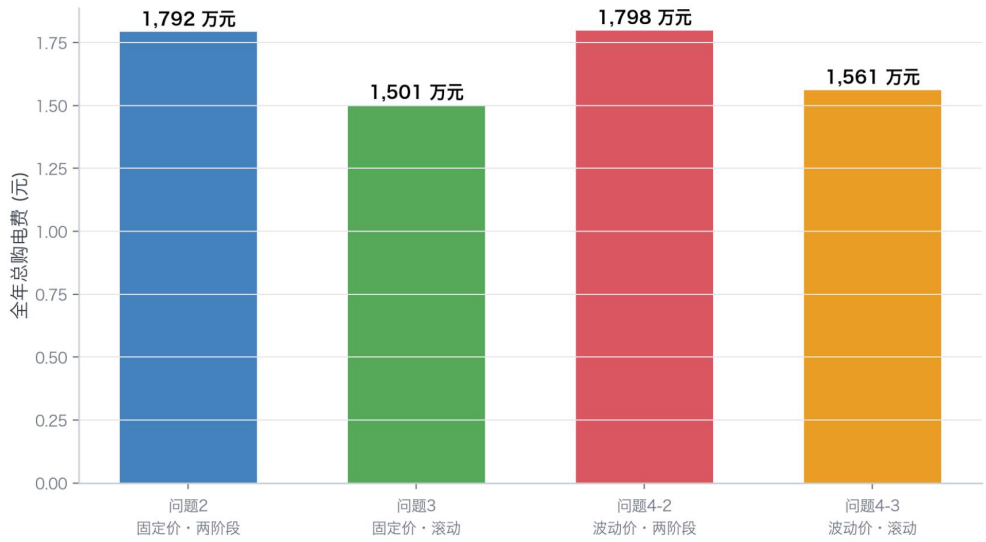


图 11　四种策略优化后成本的统计对比

表 7　四种策略费用汇总对比（单位：万元）

策略	计划购电费	调整相关费	紧急购电费	总购电费	紧急占比
问题 2 固定 · 两阶段	1654.73	–	137.61	1792.34	7.68%
问题 3 固定 · 滚动	1187.78	67.86	245.44	1501.08	16.35%
问题 4-2 波动 · 两阶段	1645.51	–	152.99	1798.50	8.51%
问题 4-3 波动 · 滚动	1244.47	71.38	245.47	1561.33	15.72%

由表 7 与图 11 可看出三点：其一，无论在固定电价还是波动电价下，滚动策略的总费用都显著低于两阶段策略；其二，波动电价使两种策略的总费用都小幅上升，但幅度都远小于策略结构之间的差异；其三，四种策略的总费用排序稳定为：问题 4-3 < 问题 3 < 问题 2 ≈ 问题 4-2，即“滚动 + 波动价”仍是全年最经济的策略。

在电价未知的情景二下，本文进一步采用 5 日滑动平均对波动电价进行预测，并据此制定计划与调整策略。电价已知与预测两种情形下的总费用对比及预测误差见附录图 18、图 19。预测结果显示：问题 4-2 的预测总费用为 1812.99 万元（较已知电价情形 1798.50 万元高 0.81%），问题 4-3 的预测总费用为 1573.89 万元（较已知电价情形 1561.33 万元高 0.80%），说明电价预测误差对总费用的影响较小，策略对电价不确定性具有较强鲁棒性。

9　模型优缺点分析

9.1　问题一

问题一模型在数据已知，且目标函数为凸性的情况和物理条件下，对于优化最优解的求解效率较高。同时由于其性质，一定可以收敛到一个全局的最优解，且对于数据结

果可靠透明（使用 DP 和贪婪算法验证）而言没有误差，除此之外，凸线性规划的结果含义基本符合“谷买峰用”的套利行为。

但模型的使用在现实数据中仍有缺陷，在实际问题中若把预测值直接作为精确值进行简单的线性规划对于实际的波动数据几乎没有鲁棒性，同时单日的储能独立和储能过程中无实际的损耗产生是不符合实际的。

9.2 问题二

经过问题一之后，问题二的模型之中引入了数据预测的要求，结合不同的报童比值选择出最合适的比值，同时结合上帝视角的实际值对于报童模型进行迭代优化升级从而使得计划购电量更贴近真实净负荷、紧急购电费尽可能低。问题二的求解模型“滑动平均预报 + 报童分位安全裕量 + 确定性 LP 计划”结构简单、无需训练、可解释性强，80 分位的安全裕量对应紧急购电 5 倍价下欠购与超购边际成本之比 4:1 的理论临界比，确定性 LP 能规范处理储能容量、充放电互斥和首末储电量一致等约束。

但其缺点同样明显，滑动平均预报滞后严重、对光伏尤其不准，统一用全年误差取分位忽略了季节与时段差异，逐时段独立取分位忽略了相邻时段误差的高度相关性，可能低估连续缺口。80% 分位仅为边际最优而非考虑储能与电价曲线后的全局最优，尾部 20% 风险全由 5 倍价紧急购电承担，且阶段一 LP 目标只含计划购电费、不含紧急购电费，会出现两阶段目标不一致的问题。

验证模型采用上帝视角的实际净负荷与 5 倍价格紧急购电事后核算缺口，优点是能直接暴露预报 + 裕量 + LP 组合的真实缺口，且逻辑清晰地量化紧急购电费占比并便于单独分析策略优劣。

9.3 问题三

问题三相比于前面的问题把一次计划升级为滚动模型预测控制，每天在 0:00、6:00、12:00、18:00 四个时点依据附件 3 逐次更新的光伏预报重新优化剩余时段决策，同时结合了插值的操作使得相关的粒度可以对齐，储能状态逐段顺延。此方法的优势在于预报越往后越准，决策也随之一次次降低相关的误差，不必在凌晨就为一整天的误差买单；结算机制区分计划购电、违约（50%）与超额（150%）并保留实时缺口五倍紧急购电，更接近实际调度规则，滚动相对“仅零点一次计划”节省的费用也能直接量化，说服力较强。

不过问题三仍未跳出确定性优化的框架：每次重规划仍把未来 24 小时预报当作确定值求解，预报本身的随机性没有进入目标函数，本质上是“确定性的滚动”而非真正的随机 MPC；并且每天需解四遍线性规划，全年计算开销明显高于前两问，50%、150% 与 5 倍等结算系数均由题面给定，模型未讨论其敏感性。

9.4 问题四

问题四把附件 4 的波动电价引入前两问的策略框架，分别重解了“波动电价 + 两阶段”与“波动电价 + 滚动”两种策略并作对比。引入分时电价后，储能得以在低价时段充

电、高价时段放电，“低充高放”的套利价值首次真正显现；再与固定电价下的问题二、问题三对照，即可把电价波动带来的成本增量与储能套利收益分开看清，为不同电价机制下的策略选择提供依据，求解框架仍沿用线性规划与报童/滚动方法，结构清晰、结果可复现。

但问题四同样继承了预报粗糙、逐时段独立取分位的缺陷，且电价与净负荷两重不确定性叠加后，仅靠负荷侧施加安全裕量更难抵御联合尾部风险；更关键的在于附件 4 的波动电价被当作已知量直接使用，既未预测电价也未对电价不确定性建模，而实际日前电价本身即有预报误差。

10 参考文献

[1]BELLMAN R. Dynamic Programming[M]. Princeton: Princeton University Press, 1957.

[2]DANTZIG G B. Linear Programming and Extensions[M]. Princeton: Princeton University Press, 1963.

[3]BOYD S, VANDENBERGHE L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004.

[4]QIN Y, WANG R, VAKHARIA A J, et al. The newsvendor problem: Review and directions for future research[J]. European Journal of Operational Research, 2011, 213(2): 361-374.

[5]SILVER E A, PYKE D F, PETERSON R. Inventory Management and Production Planning and Scheduling[M]. 3rd ed. New York: Wiley, 1998.

[6]CAMACHO E F, BORDONS C. Model Predictive Control[M]. 2nd ed. London: Springer, 2007.

[7]MAYNE D Q, RAWLINGS J B, RAO C V, et al. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789-814.

[8]KHAN A A, NAEEM M, IQBAL M. A compendium of optimization objectives, constraints, tools and algorithms for energy management in microgrids[J]. Renewable and Sustainable Energy Reviews, 2016, 58: 1664-1683.

[9]HIRSCH A, PARAG Y, GUERRERO J. Microgrids: A review of technologies, key drivers, and outstanding issues[J]. Renewable and Sustainable Energy Reviews, 2018, 90: 402-411.

[10MORSTYN T, HREDZAK B, AGUILERA R P, et al. Model predictive control for distributed microgrid battery energy storage systems[J]. IEEE Transactions on Control Systems Technology, 2018, 26(3): 1107-1114.

[11]GARCIA-TORRES F, BORDONS C. Optimal economical schedule of hydrogen-based microgrids with hybrid storage using model predictive control[J]. IEEE Transactions on Industrial Electronics, 2015, 62(8): 5195-5207.

[12]CHENG T, ZENG L, TANG H, et al. Optimal dispatch of integrated energy system considering time-space dimensional demand response based on time-of-use electricity price[J]. Frontiers in Energy Research, 2025, 13: 1540071.

A　图像补充

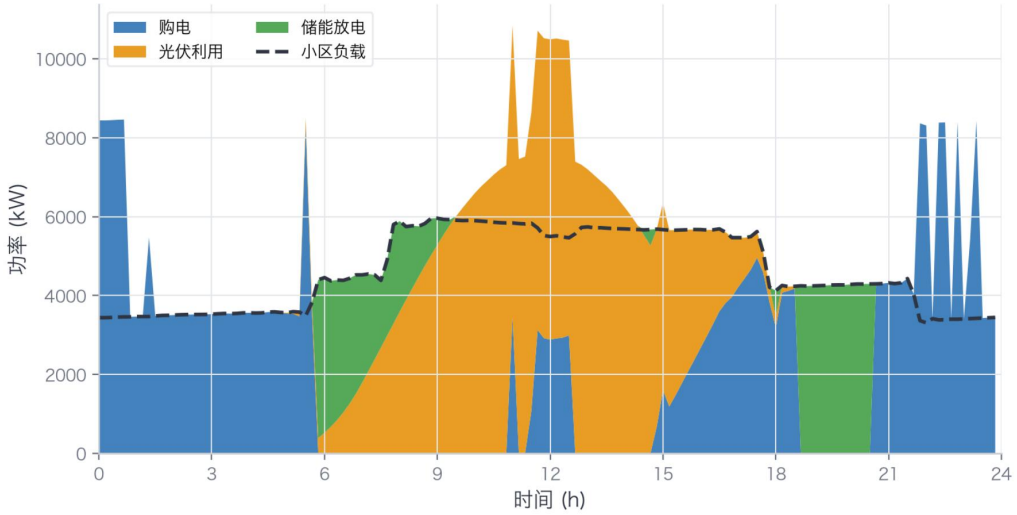


图 12　各时段功率构成的堆叠关系图

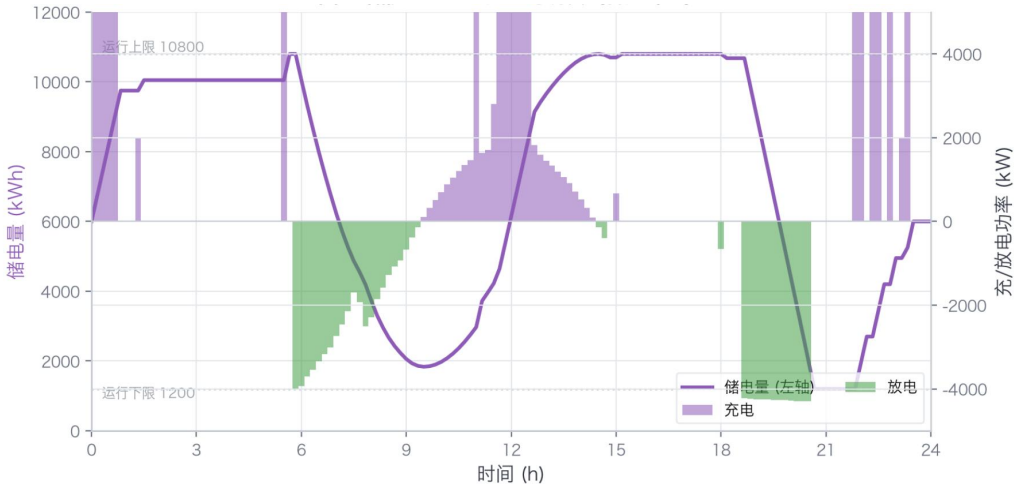


图 13　问题一储能 SOC 与充放电调度（谷充峰放）

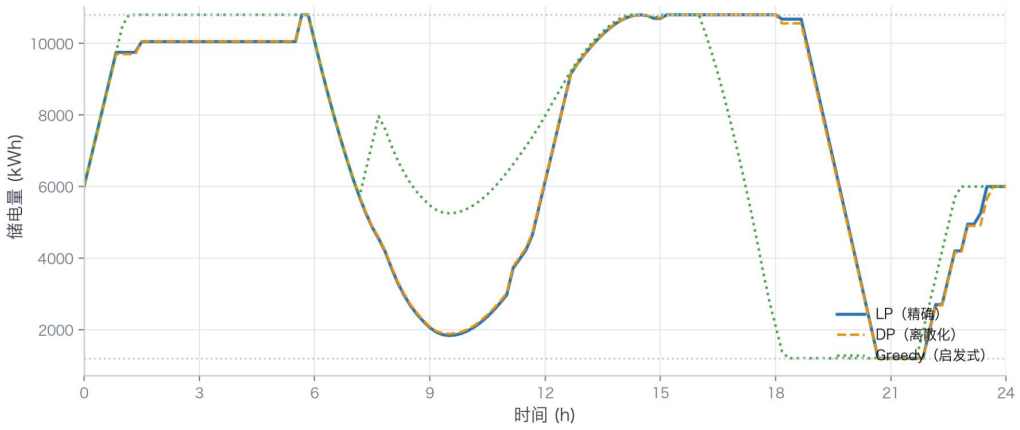


图 14　储能 SOC 轨迹对比（三算法）

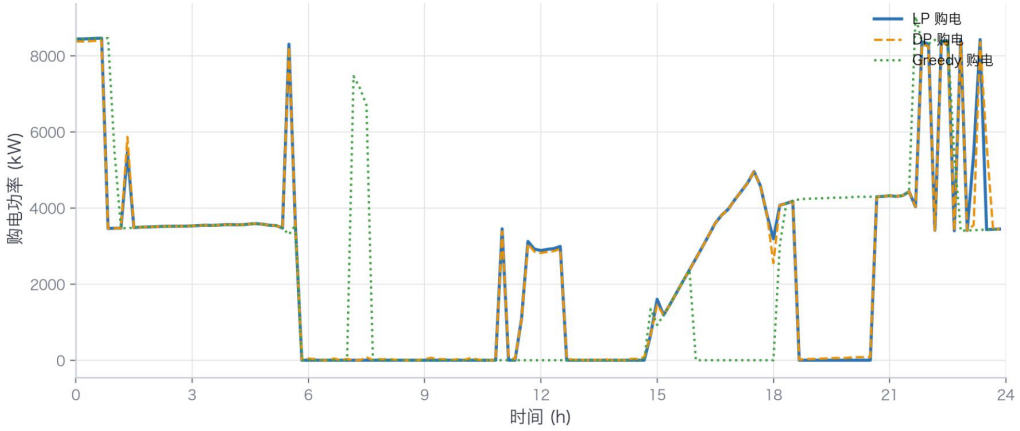


图 15　购电策略对比（三算法）

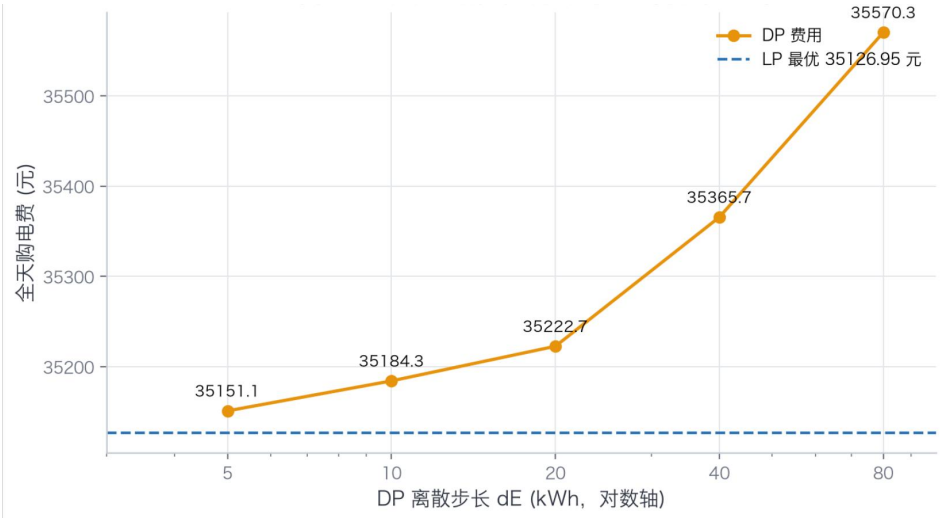


图 16　DP 随步长细化单调收敛到 LP（最优性验证）

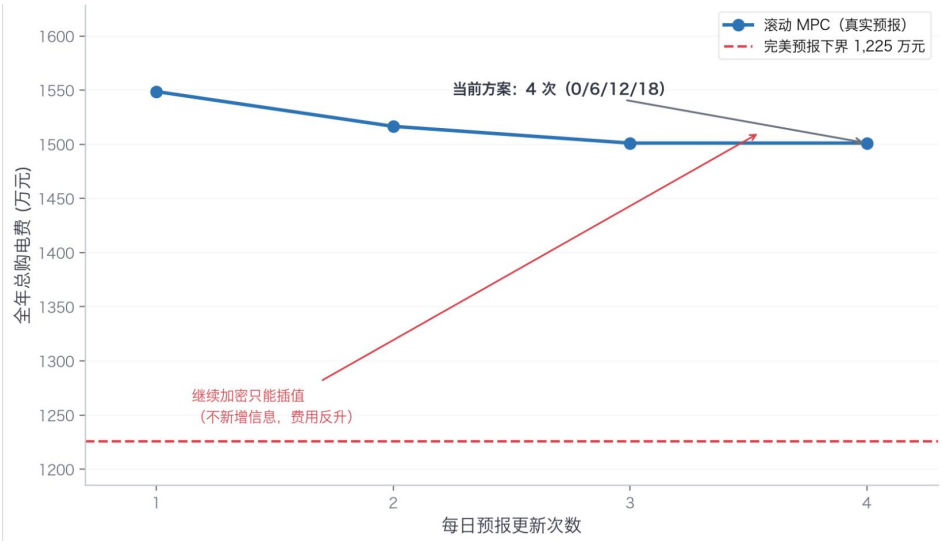


图 17　预报更新频率敏感性（问题三滚动价值验证）

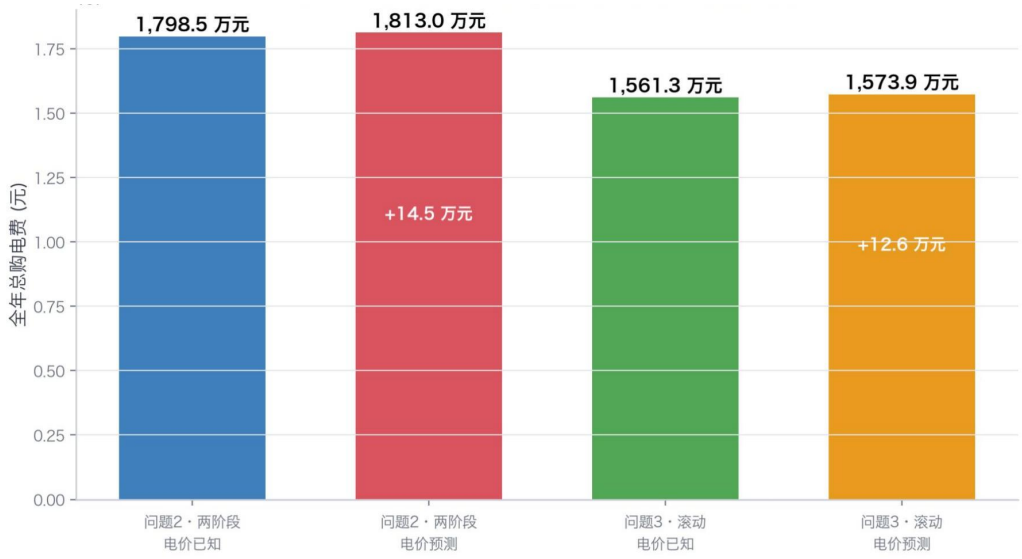


图 18　电价已知与预测情形下总费用对比（问题四）

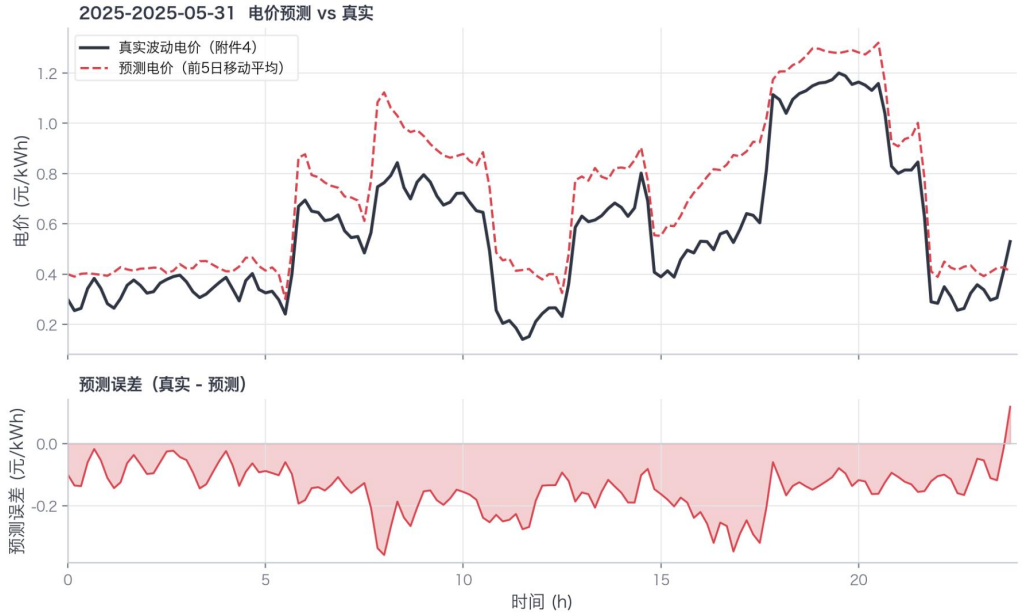


图 19　电价预测误差统计（问题四）

B　各问题求解、验证与绘图代码

本附录给出论文全部可运行代码，按“公共模块 + 各问题”组织；每个问题内按 `solve`（求解）、`verify`（验证）、`plot`（绘图）三类分别列出。代码与论文图表一一对应，可直接复现文中的求解结果与全部图形。

B.1　公共模块 `common.py`

```python
# -*- coding: utf-8 -*-
import os
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy import sparse

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = "/Users/jimfang/Desktop/数学建模比赛/2026数学建模国赛/
    CUMCM2026Problems/C题/附件"

E_MAX   = 12000.0
E_MIN   = 1200.0
E_UPPER = 10800.0
E_LOWER = 1200.0
```

```
P_MAX   = 5000.0
ETA     = 0.90
E0      = 6000.0
DT      = 1.0 / 6.0
N       = 144

PALETTE = {
    "blue":   "#2E74B5",
    "red":    "#D64550",
    "orange": "#E8930C",
    "green":  "#43A047",
    "purple": "#8E5BB8",
    "teal":   "#159A9C",
    "gray":   "#9AA0A6",
    "dark":   "#2F3542",
}


def hhmm(minutes):
    d = minutes // (24 * 60)
    m = minutes % (24 * 60)
    h, mm = m // 60, m % 60
    return f"{h}:{mm:02d}" + (f"+{d}" if d else "")


def period_label(t):
    return f"{hhmm(t * 10)}-{hhmm((t + 1) * 10)}"


def period_labels_all():
    return [period_label(t) for t in range(N)]


def load_fj1():
    df = pd.read_excel(os.path.join(DATA_DIR, "附件1.xlsx"), header
   =0)
    price = df.iloc[:, 1].astype(float).to_numpy()
    load  = df.iloc[:, 2].astype(float).to_numpy()
    pv    = df.iloc[:, 3].astype(float).to_numpy()
    return price, load, pv

```

```
55
56 def _read_daily_matrix(sheet):
57     df = pd.read_excel(os.path.join(DATA_DIR, sheet), header=0)
58     dates = df.iloc[:, 0].to_numpy()
59     mat   = df.iloc[:, 1:1 + N].astype(float).to_numpy()
60     return dates, mat
61
62
63 def load_fj2():
64     dates, load = _read_daily_matrix("附件2.xlsx")
65     pv = pd.read_excel(os.path.join(DATA_DIR, "附件2.xlsx"),
66                        sheet_name="光伏发电实际功率", header=0)
67     pv_mat = pv.iloc[:, 1:1 + N].astype(float).to_numpy()
68     return dates, load, pv_mat
69
70
71 def load_fj3():
72     df = pd.read_excel(os.path.join(DATA_DIR, "附件3.xlsx"), header
    =0)
73     df = df.copy()
74     df.iloc[:, 0] = df.iloc[:, 0].ffill()
75     dates = df.iloc[:, 0].to_numpy()[::4]
76     vals  = df.iloc[:, 2:2 + 24].astype(float).to_numpy()
77     n_days = vals.shape[0] // 4
78     fc = vals[:n_days * 4].reshape(n_days, 4, 24)
79     return dates[:n_days], fc
80
81
82 def load_fj4():
83     dates, price = _read_daily_matrix("附件4.xlsx")
84     return dates, price
85
86
87 def solve_dispatch(price, load, pv, E_start, E_end, w_buy=1.0,
    b_fix=None, v_end=None):
88     """单日线性规划调度。
89
90     E_end=None 时不再强制 24:00 储电量归位（跨日连续），末端储电量按残值
    v_end
91     （元/kWh）计价，避免 LP 把储电量人为抽干到下限。
92     """
```

```
    price = np.asarray(price, float)
    load  = np.asarray(load, float)
    pv    = np.asarray(pv, float)
    net   = load - pv
    n = len(load)

    # 变量顺序: u / ch / dis / s / E, 各 n 个
    i_u   = 0
    i_ch  = n
    i_dis = 2 * n
    i_s   = 3 * n
    i_E   = 4 * n
    nvar  = 5 * n

    c = np.zeros(nvar)
    c[i_u:i_u + n] = w_buy * price * DT
    if v_end is not None:
        c[i_E + n - 1] = -float(v_end)

    n_eq = 2 * n + (1 if E_end is not None else 0)
    A_eq = np.zeros((n_eq, nvar))
    b_eq = np.zeros(n_eq)
    row = 0
    for t in range(n):
        A_eq[row, i_u + t]   = 1.0
        A_eq[row, i_s + t]   = -1.0
        A_eq[row, i_dis + t] = 1.0
        A_eq[row, i_ch + t]  = -1.0
        b_eq[row] = net[t] - (b_fix[t] if b_fix is not None else
0.0)
        row += 1
    for t in range(n):
        A_eq[row, i_E + t] = 1.0
        if t > 0:
            A_eq[row, i_E + t - 1] = -1.0
        else:
            b_eq[row] += E_start
        A_eq[row, i_ch + t]  = -ETA * DT
        A_eq[row, i_dis + t] = DT / ETA
        row += 1
    if E_end is not None:
```

```
        A_eq[row, i_E + n - 1] = 1.0
        b_eq[row] = E_end

    lb = np.zeros(nvar)
    ub = np.full(nvar, np.inf)
    lb[i_ch:i_ch + n] = 0.0;   ub[i_ch:i_ch + n] = P_MAX
    lb[i_dis:i_dis + n] = 0.0; ub[i_dis:i_dis + n] = P_MAX
    lb[i_s:i_s + n] = 0.0;     ub[i_s:i_s + n] = pv
    lb[i_E:i_E + n] = E_LOWER; ub[i_E:i_E + n] = E_UPPER

    res = linprog(c, A_eq=sparse.csr_matrix(A_eq), b_eq=b_eq,
                  bounds=list(zip(lb, ub)), method="highs")

    if not res.success:
        raise RuntimeError(f"LP 求解失败: {res.message}")

    x = res.x
    u   = x[i_u:i_u + n]
    ch  = x[i_ch:i_ch + n]
    dis = x[i_dis:i_dis + n]
    s   = x[i_s:i_s + n]
    E   = x[i_E:i_E + n]
    b   = u if b_fix is None else (b_fix + u)

    cost = float(np.sum(w_buy * price * u) * DT)
    return dict(b=b, ch=ch, dis=dis, s=s, E=E, u=u, cost=cost)


def solve_plan(price, load, pv, E_start=E0, E_end=E0, v_end=None):
    return solve_dispatch(price, load, pv, E_start, E_end, w_buy
   =1.0, v_end=v_end)


def salvage_value(price, w_buy=1.0):
    """跨日连续储能的末端残值（元/kWh）：留存 1 kWh 储能，次日可放电 ETA
   kWh，
    其价值 = 次日最低购电价 × 效率（水价值下界），即"用最便宜的电买回"的成本。

    不能用均价：均价 > 谷价时会把末端储能人为高估，诱导 LP 在谷时买电充到上
   限、
    午夜套取残值（"套利泵"），储能被钉在 E_UPPER 且计划购电费虚高。
```

```
    """
    return float(w_buy * ETA * np.min(price))


def solve_plan_carry(price, load, pv, E_start):
    """跨日连续调度（问题2/3/4）：24:00 储电量自由，按残值计价，不再日末归位E0。"""
    return solve_dispatch(price, load, pv, E_start, None, w_buy=1.0,
                          v_end=salvage_value(price))


def warmup_storage(price_daily, load, pv, n_warm, E_start=E0):
    """从 1 月 1 日 0:00=E_start 起，按已知负荷/光伏连续演化 n_warm 天，
    返回第 n_warm 天 0:00 的储电量（用于给出 2 月 1 日 0:00 的真实初值）。
    price_daily(d) 返回第 d 天 144 维电价。"""
    E_cur = float(E_start)
    for d in range(n_warm):
        res = solve_plan_carry(price_daily(d), load[d], pv[d], E_cur)
        E_cur = float(res["E"][-1])
    return E_cur


def solve_emergency(price, load, pv, b_plan, E_start=E0, E_end=E0, w_buy=5.0):
    price = np.asarray(price, float)
    load  = np.asarray(load, float)
    pv    = np.asarray(pv, float)
    b_plan = np.asarray(b_plan, float)
    net = load - pv
    n = len(load)

    i_e   = 0
    i_ch  = n
    i_dis = 2 * n
    i_c   = 3 * n
    i_E   = 4 * n
    nvar  = 5 * n

    c_obj = np.zeros(nvar)
```

```
    c_obj[i_e:i_e + n] = w_buy * price * DT

    n_eq = 2 * n + 1
    A_eq = np.zeros((n_eq, nvar)); b_eq = np.zeros(n_eq)
    row = 0
    for t in range(n):
        A_eq[row, i_e + t]   = 1.0
        A_eq[row, i_c + t]   = -1.0
        A_eq[row, i_dis + t] = 1.0
        A_eq[row, i_ch + t]  = -1.0
        b_eq[row] = net[t] - b_plan[t]
        row += 1
    for t in range(n):
        A_eq[row, i_E + t] = 1.0
        if t > 0:
            A_eq[row, i_E + t - 1] = -1.0
        else:
            b_eq[row] += E_start
        A_eq[row, i_ch + t]  = -ETA * DT
        A_eq[row, i_dis + t] = DT / ETA
        row += 1
    A_eq[row, i_E + n - 1] = 1.0; b_eq[row] = E_end

    lb = np.zeros(nvar); ub = np.full(nvar, np.inf)
    lb[i_ch:i_ch + n] = 0.0;   ub[i_ch:i_ch + n] = P_MAX
    lb[i_dis:i_dis + n] = 0.0; ub[i_dis:i_dis + n] = P_MAX
    lb[i_c:i_c + n] = 0.0;     ub[i_c:i_c + n] = np.inf
    lb[i_E:i_E + n] = E_LOWER; ub[i_E:i_E + n] = E_UPPER

    res = linprog(c_obj, A_eq=sparse.csr_matrix(A_eq), b_eq=b_eq,
                  bounds=list(zip(lb, ub)), method="highs")
    if not res.success:
        raise RuntimeError(f"紧急购电 LP 求解失败: {res.message}")

    x = res.x
    e   = x[i_e:i_e + n]
    ch  = x[i_ch:i_ch + n]
    dis = x[i_dis:i_dis + n]
    c   = x[i_c:i_c + n]
    E   = x[i_E:i_E + n]
    b   = b_plan + e
```

```
    cost = float(np.sum(w_buy * price * e) * DT)
    return dict(e=e, b=b, ch=ch, dis=dis, c=c, E=E, cost=cost)


def forecast_to_10min(fc24, issue_hour, pv_at_issue=0.0):
    knots_h = np.arange(issue_hour, issue_hour + 25, dtype=float)
    knots_v = np.concatenate([[pv_at_issue], np.asarray(fc24, float
    )])
    t10 = np.arange(N) / 6.0
    pv10 = np.interp(t10, knots_h, knots_v)
    return np.clip(pv10, 0.0, None)


def setup_plot():
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({
        "font.sans-serif": ["Hiragino Sans GB", "Arial Unicode MS",
    "Songti SC", "SimHei"],
        "axes.unicode_minus": False,
        "figure.dpi": 120,
        "savefig.dpi": 200,
        "savefig.bbox": "tight",
        "figure.facecolor": "white",
        "savefig.facecolor": "white",
        "axes.facecolor": "white",
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.edgecolor": "#C7CCD4",
        "axes.linewidth": 0.9,
        "axes.grid": True,
        "grid.color": "#E4E7EB",
        "grid.linewidth": 0.7,
        "grid.alpha": 1.0,
        "axes.titlesize": 12,
        "axes.titleweight": "bold",
        "axes.titlecolor": "#2F3542",
        "axes.labelsize": 11,
        "axes.labelcolor": "#2F3542",
        "xtick.labelsize": 9,
```

```
        "ytick.labelsize": 9,
        "xtick.color": "#6B7280",
        "ytick.color": "#6B7280",
        "legend.fontsize": 9,
        "legend.frameon": False,
    })
    return plt


def despine(ax):
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)
    return ax


def merge_contiguous(quantities, dt=DT):
    intervals = []
    n = len(quantities)
    t = 0
    while t < n:
        if quantities[t] > 1e-9:
            start = t
            acc = 0.0
            while t < n and quantities[t] > 1e-9:
                acc += quantities[t] * dt
                t += 1
            intervals.append((start, t, acc))
        else:
            t += 1
    return intervals


def aggregate_4h(x):
    out = np.zeros(6)
    for p in range(6):
        out[p] = np.sum(x[p * 24:(p + 1) * 24]) * DT
    return out


FOUR_H_LABELS = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
                 "12:00-16:00", "16:00-20:00", "20:00-24:00"]
```

```
329
330
331 def write_plan_sheet(df_by_day, path, sheet="计划购电量"):
332     dates = sorted(df_by_day.keys())
333     cols = ["日期\\时间"] + period_labels_all() + ["全天购电量", "全天
    购电费"]
334     rows = []
335     for d in dates:
336         rec = df_by_day[d]
337         buy_kwh = np.round(rec["b"] * DT, 4)
338         total_kwh = round(float(np.sum(buy_kwh)), 4)
339         total_fee = round(float(np.sum(rec["price"] * rec["b"]) *
    DT), 2)
340         rows.append([_fmt_date(d)] + list(buy_kwh) + [total_kwh,
    total_fee])
341     out = pd.DataFrame(rows, columns=cols)
342     _append_sheet(out, path, sheet)
343     return out
344
345
346 def write_emergency_sheet(emergency_by_day, path, sheet="紧急购电量")
    :
347     dates = sorted(emergency_by_day.keys())
348     rows = []
349     for d in dates:
350         e = np.asarray(emergency_by_day[d])
351         inter = merge_contiguous(e)
352         if not inter:
353             rows.append([_fmt_date(d), "", ""])
354             continue
355         for (s, t, kwh) in inter:
356             label = f"{period_label(s).split('-')[0]}-{period_label
    (t - 1).split('-')[1]}"
357             rows.append([_fmt_date(d), label, round(kwh, 4)])
358     out = pd.DataFrame(rows, columns=["日期", "购电时间段", "购电量"])
359     _append_sheet(out, path, sheet)
360     return out
361
362
363 def write_charge_sheet(charge_by_day, path, sheet="充放电量"):
364     dates = sorted(charge_by_day.keys())
```

```
    rows = []
    for d in dates:
        rec = charge_by_day[d]
        ch4, dis4 = aggregate_4h(rec["ch"]), aggregate_4h(rec["dis"])
        date_str = _fmt_date(d)
        for p in range(6):
            time_col = "0:00" if p == 0 else ("24:00" if p == 1 else "")
            e_col = round(rec["E_start"], 1) if p == 0 else (round(rec["E_end"], 1) if p == 1 else "")
            rows.append([date_str if p == 0 else "", FOUR_H_LABELS[p],
                         round(ch4[p], 2), round(dis4[p], 2), time_col, e_col])
    out = pd.DataFrame(rows, columns=["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
    _append_sheet(out, path, sheet)
    return out


def _fmt_date(d):
    return str(pd.Timestamp(d).date())


def _autofit(ws, sample=120, min_w=9, max_w=26):
    for col_cells in ws.columns:
        letter = col_cells[0].column_letter
        if letter == "A":
            ws.column_dimensions[letter].width = 12.0
            continue
        max_len = 0
        for i, cell in enumerate(col_cells):
            if i > sample:
                break
            if cell.value is not None:
                max_len = max(max_len, len(str(cell.value)))
        ws.column_dimensions[letter].width = min(max(max_len + 1.5, min_w), max_w)
```

```
def _append_sheet(df, path, sheet):
    if os.path.exists(path):
        with pd.ExcelWriter(path, engine="openpyxl", mode="a",
   if_sheet_exists="replace") as w:
            df.to_excel(w, sheet_name=sheet, index=False)
            _autofit(w.sheets[sheet])
    else:
        with pd.ExcelWriter(path, engine="openpyxl") as w:
            df.to_excel(w, sheet_name=sheet, index=False)
            _autofit(w.sheets[sheet])


def write_summary(df, path):
    df.to_csv(path, index=False, encoding="utf-8-sig")
```

Listing 1　common.py（储能参数、数据读取、LP 求解器与绘图工具）

B.2　问题 1 代码

B.2.1　solve ——问题 1 求解

```
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
   __file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))

price, load, pv = C.load_fj1()

res = C.solve_plan(price, load, pv, E_start=C.E0, E_end=C.E0)
b, ch, dis, s, E = res["b"], res["ch"], res["dis"], res["s"], res["
   E"]
cost = float(np.sum(price * b) * C.DT)
buy_kwh = float(np.sum(b) * C.DT)

b_none = np.maximum(load - pv, 0.0)
```

```
cost_none = float(np.sum(price * b_none) * C.DT)

spec_idx = [60, 72, 84, 96, 108, 120]
print("问题1 结果（LP 精确解）")
print("表1  微网在指定时间段的购电量及全天购电量/购电费")
print(f"{'时间段':<14}{'购电量(kWh)':>14}")
for t in spec_idx:
    print(f"{C.period_label(t):<16}{b[t] * C.DT:>12.4f}")
print(f"{'全天购电量':<14}{buy_kwh:>14.4f}")
print(f"{'全天购电费':<14}{cost:>14.4f} 元")

ch4, dis4 = C.aggregate_4h(ch), C.aggregate_4h(dis)
print()
print("表2  储能设备在指定时间段的充放电量及储电量")
print(f"{'时间段':<14}{'充电量(kWh)':>14}{'放电量(kWh)':>14}")
for p in range(6):
    print(f"{C.FOUR_H_LABELS[p]:<16}{ch4[p]:>12.4f}{dis4[p]:>12.4f}
    ")
print(f"{'0:00储电量':<14}{C.E0:>14.2f}")
print(f"{'24:00储电量':<14}{E[-1]:>14.2f}")

print()
print("交叉验证")
print(f"  无储能基线购电费 : {cost_none:.4f} 元")
print(f"  储能(DP/LP)购电费 : {cost:.4f} 元")
print(f"  储能相对无储能节省: {cost_none - cost:.4f} 元")
print(f"  弃光电量合计       : {float(np.sum(s) * C.DT):.4f} kWh")
print(f"  末端储电量(应=6000): {E[-1]:.4f} kWh")

res1 = pd.DataFrame({
    "时间段": C.period_labels_all(),
    "购电量": b * C.DT,
})
res1.to_excel(os.path.join(HERE, "result1.xlsx"), sheet_name="计划购
    电量", index=False)

res2 = pd.DataFrame({
    "时间段": C.FOUR_H_LABELS,
    "充电量": ch4,
    "放电量": dis4,
    "时刻": ["0:00", "24:00", "", "", "", ""],
```

```
    "储电量": [C.E0, E[-1], "", "", "", ""],
})
with pd.ExcelWriter(os.path.join(HERE, "result1.xlsx"), engine="openpyxl",
                    mode="a", if_sheet_exists="replace") as w:
    res2.to_excel(w, sheet_name="充放电量", index=False)

print(f"\n已保存 result1.xlsx 到 {HERE}")

np.savez(os.path.join(HERE, "_q1_data.npz"),
         price=price, load=load, pv=pv, b=b, ch=ch, dis=dis, s=s, E=E)
print("已保存绘图数据 _q1_data.npz")
```

Listing 2　solve_q1.py（问题 1 线性规划求解）

B.2.2　verify ——问题 1 验证

```
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import common as C
from common import DT, N, E0, E_LOWER, E_UPPER, P_MAX, ETA

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()

price, load, pv = C.load_fj1()
t = np.arange(N) / 6.0
P = C.PALETTE
CURT = {"LP": P["blue"], "DP": P["orange"], "Greedy": P["green"]}


def lp_dispatch():
    res = C.solve_plan(price, load, pv, E0, E0)
```

```
    return res["b"], res["ch"], res["dis"], np.concatenate([[E0], res["E"]]), res["cost"]


def dp_dispatch(dE=20.0):
    E_grid = np.arange(E_LOWER, E_UPPER + 1e-9, dE)
    M = len(E_grid) - 1
    i0 = int(round((E0 - E_LOWER) / dE))
    assert abs(E_grid[i0] - E0) < 1e-6, "E0 未落在网格上"

    # D[i,j] = E_i - E_j，充电/放电功率由 ΔE 反推
    D = E_grid[:, None] - E_grid[None, :]
    ch_mat = np.where(D > 0, D / (ETA * DT), 0.0)
    dis_mat = np.where(D < 0, -D * ETA / DT, 0.0)
    feas_base = (ch_mat <= P_MAX + 1e-9) & (dis_mat <= P_MAX + 1e-9)

    INF = 1e18
    dp = np.full(M + 1, INF); dp[i0] = 0.0
    pred = np.zeros((N, M + 1), dtype=np.int64)

    for tt in range(N):
        feas = feas_base & (dis_mat <= load[tt] + 1e-9)
        b_mat = np.maximum(0.0, load[tt] - pv[tt] + ch_mat - dis_mat)
        Cmat = np.where(feas, price[tt] * b_mat * DT, INF)
        cand = dp[None, :] + Cmat
        dp = cand.min(axis=1)
        pred[tt] = cand.argmin(axis=1)

    cost = dp[i0]
    E_traj = np.zeros(N); i = i0
    for tt in range(N - 1, -1, -1):
        E_traj[tt] = E_grid[i]; i = pred[tt, i]
    E_full = np.concatenate([[E0], E_traj])
    dEseq = E_traj - E_full[:-1]
    ch = np.where(dEseq > 0, dEseq / (ETA * DT), 0.0)
    dis = np.where(dEseq < 0, -dEseq * ETA / DT, 0.0)
    b = np.maximum(0.0, load - pv + ch - dis)
    return b, ch, dis, E_full, cost

```

```

def greedy_dispatch():
    threshold = np.median(price)
    E = E0
    ch = np.zeros(N); dis = np.zeros(N); b = np.zeros(N); s = np.
   zeros(N)
    for tt in range(N):
        net = load[tt] - pv[tt]
        if net <= 0:
            excess = -net
            c = min(excess, P_MAX, (E_UPPER - E) / (ETA * DT))
            ch[tt] = c; s[tt] = excess - c
        elif price[tt] < threshold:
            c = min(P_MAX, (E_UPPER - E) / (ETA * DT))
            ch[tt] = c; b[tt] = net + c
        else:
            d = min(net, P_MAX, (E - E_LOWER) / (DT / ETA))
            dis[tt] = d; b[tt] = net - d
        E = E + ETA * ch[tt] * DT - dis[tt] * DT / ETA

    excess = E - E0
    if excess > 0:
        rem = excess
        for tt in range(N - 1, -1, -1):
            if rem <= 1e-6:
                break
            drop = min(ETA * ch[tt] * DT, rem)
            ch[tt] -= drop / (ETA * DT)
            rem -= drop
    elif excess < 0:
        rem = -excess
        for tt in range(N - 1, -1, -1):
            if rem <= 1e-6:
                break
            drop = min(dis[tt] * DT / ETA, rem)
            dis[tt] -= drop / (DT / ETA)
            rem -= drop

    b = np.maximum(0.0, load - pv + ch - dis)
    E_hist = np.zeros(N + 1); E_hist[0] = E0; Ee = E0
    for tt in range(N):
```

```
        Ee = Ee + ETA * ch[tt] * DT - dis[tt] * DT / ETA
        E_hist[tt + 1] = Ee
    assert (abs(E_hist[-1] - E0) < 0.5 and E_hist.min() >= E_LOWER
    - 1e-6
            and E_hist.max() <= E_UPPER + 1e-6), "贪婪解末端电量/SOC
    越界"
    cost = float(np.sum(price * b) * DT)
    return b, ch, dis, s, E_hist, cost


b_lp, ch_lp, dis_lp, E_lp, cost_lp = lp_dispatch()
b_dp, ch_dp, dis_dp, E_dp, cost_dp = dp_dispatch(dE=20.0)
b_g, ch_g, dis_g, s_g, E_g, cost_g = greedy_dispatch()

print("问题1 三算法交叉验证（全天购电费，元）")
print(f"  线性规划 LP    : {cost_lp:>12.4f}    (精确最优)")
print(f"  动态规划 DP    : {cost_dp:>12.4f}    (离散化 dE=20 kWh)")
print(f"  贪婪算法 Greedy: {cost_g:>12.4f}    (启发式上界)")
print(f"  |DP - LP|      = {abs(cost_dp - cost_lp):.4f} 元  "
      f"({abs(cost_dp - cost_lp) / cost_lp * 100:.4f}%)")
print(f"  Greedy - LP    = {cost_g - cost_lp:.4f} 元  "
      f"({(cost_g - cost_lp) / cost_lp * 100:.3f}%，应为非负，证明 LP
    可行)")
print(f"  末端电量核对   : LP={E_lp[-1]:.2f}  DP={E_dp[-1]:.2f}
    Greedy={E_g[-1]:.2f}  (均应=6000)")
print(f"  SOC 区间       : Greedy min={E_g.min():.1f} / max={E_g.max
    ():.1f}  (界=[1200,10800])")
print(f"  全天购电量     : LP={np.sum(b_lp) * DT:.1f} kWh, "
      f"DP={np.sum(b_dp) * DT:.1f} kWh, Greedy={np.sum(b_g) * DT:.1
    f} kWh")
print()
print("结论：DP 与 LP 几乎一致 → LP 解为全局最优；")
print("      Greedy 费用略高但同量级 → LP 费用物理可实现，未低估。")

fig, ax = plt.subplots(figsize=(7, 4.4))
labels = ["线性规划 LP\n(精确)", "动态规划 DP\n(离散化)", "贪婪算法
    Greedy\n(启发式)"]
vals = [cost_lp, cost_dp, cost_g]
colors = [CURT["LP"], CURT["DP"], CURT["Greedy"]]
bars = ax.bar(labels, vals, color=colors, alpha=0.88, width=0.55)
for b_bar, v in zip(bars, vals):
```

```
    ax.text(b_bar.get_x() + b_bar.get_width() / 2, v + vals[0] * 0.006,
            f"{v:.2f}", ha="center", fontsize=11, weight="bold")
ax.set_ylabel("全天购电费（元）")
ax.set_ylim(0, max(vals) * 1.15)
ax.set_title("验证图1　三种算法购电费对比（交叉验证）", fontsize=12, weight="bold")
ax.grid(alpha=0.3, axis="y")
fig.tight_layout()
fig.savefig(os.path.join(OUT, "验证图1_三算法购电费对比.png"))
plt.close(fig)

fig, ax = plt.subplots(figsize=(9.5, 4.4))
ax.plot(np.arange(N + 1) / 6.0, E_lp, color=CURT["LP"], lw=2, label="LP（精确）")
ax.plot(np.arange(N + 1) / 6.0, E_dp, color=CURT["DP"], lw=1.6, ls="--", label="DP（离散化）")
ax.plot(np.arange(N + 1) / 6.0, E_g, color=CURT["Greedy"], lw=1.6, ls=":", label="Greedy（启发式）")
ax.axhline(E_LOWER, color="#C7CCD4", ls=":", lw=1.2); ax.axhline(E_UPPER, color="#C7CCD4", ls=":", lw=1.2)
ax.set_xlabel("时间（h）"); ax.set_ylabel("储电量（kWh）")
ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
ax.margins(x=0)
ax.set_title("验证图2　储能 SOC 轨迹对比（三算法）", fontsize=12, weight="bold")
ax.legend(loc="lower right")
fig.tight_layout()
fig.savefig(os.path.join(OUT, "验证图2_储能SOC轨迹对比.png"))
plt.close(fig)

fig, ax = plt.subplots(figsize=(9.5, 4.4))
ax.plot(t, b_lp, color=CURT["LP"], lw=2, label="LP 购电")
ax.plot(t, b_dp, color=CURT["DP"], lw=1.5, ls="--", label="DP 购电")
ax.plot(t, b_g, color=CURT["Greedy"], lw=1.5, ls=":", label="Greedy 购电")
ax.set_xlabel("时间（h）"); ax.set_ylabel("购电功率（kW）")
ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
ax.margins(x=0)
ax.set_title("验证图3　购电策略对比（三算法）", fontsize=12, weight="bold")
```

```
ax.legend(loc="upper right")
fig.tight_layout()
fig.savefig(os.path.join(OUT, "验证图3_购电策略对比.png"))
plt.close(fig)

dEs = np.array([80.0, 40.0, 20.0, 10.0, 5.0])
dp_costs = np.array([dp_dispatch(de)[4] for de in dEs])
rel_err = (dp_costs - cost_lp) / cost_lp * 100

fig, ax = plt.subplots(figsize=(7.5, 4.4))
ax.plot(dEs, dp_costs, "o-", color=CURT["DP"], lw=1.8, ms=6, label="DP 费用")
ax.axhline(cost_lp, color=CURT["LP"], lw=1.5, ls="--", label=f"LP 最优 {cost_lp:.2f} 元")
ax.set_xscale("log"); ax.set_xlim(3, 100)
ax.set_xticks(dEs); ax.set_xticklabels([f"{int(d)}" for d in dEs])
ax.set_xlabel("DP 离散步长 dE (kWh，对数轴)")
ax.set_ylabel("全天购电费 (元)")
for de, c in zip(dEs, dp_costs):
    ax.annotate(f"{c:.1f}", (de, c), textcoords="offset points", xytext=(0, 8),
                ha="center", fontsize=8.5)
ax.set_title("验证图4  DP 随步长细化单调收敛到 LP（最优性验证）", fontsize=12, weight="bold")
ax.legend(loc="upper right")
fig.tight_layout()
fig.savefig(os.path.join(OUT, "验证图4_DP收敛性.png"))
plt.close(fig)

print(f"\nDP 收敛（dE -> 0）:")
for de, c, r in zip(dEs, dp_costs, rel_err):
    print(f"   dE={de:5.0f} kWh  ->  {c:10.2f} 元  (相对 LP +{r:7.4f}%)")
print(f"\n四张验证图已生成到 {OUT}")
```

Listing 3　verify_q1.py（问题 1 DP/Greedy 双重验证）

B.2.3　plot ——问题 1 绘图

```
# -*- coding: utf-8 -*-
import os
import sys
```

```
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()
from matplotlib.patches import FancyBboxPatch

d = np.load(os.path.join(HERE, "_q1_data.npz"))
price, load, pv = d["price"], d["load"], d["pv"]
b, ch, dis, s, E = d["b"], d["ch"], d["dis"], d["s"], d["E"]
t = np.arange(C.N) / 6.0
pv_used = pv - s
P = C.PALETTE


def fig1_topo():
    fig, ax = plt.subplots(figsize=(9, 5.4))
    ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 6.6)

    C_PV, C_GRID, C_STO, C_LOAD, C_BUS = P["orange"], P["blue"], P["purple"], P["red"], "#5A6472"

    ax.plot([2.0, 6.7], [3.0, 3.0], color=C_BUS, lw=4.5, solid_capstyle="round", zorder=2)
    ax.text(4.35, 3.32, "微网母线", ha="center", fontsize=10.5, color=C_BUS, weight="bold")

    def node(x, y, label, sub="", color=C_GRID, w=2.0, h=1.15):
        ax.add_patch(FancyBboxPatch((x - w / 2, y - h / 2), w, h,
                     boxstyle="round,pad=0.08", fc=color, ec="none", lw=0, alpha=0.94))
        ax.text(x, y + 0.14, label, ha="center", va="center", fontsize=11.5,
                color="white", weight="bold")
        if sub:
            ax.text(x, y - 0.30, sub, ha="center", va="center",
```

```
    fontsize=8.5, color="white")

    def arrow(x0, y0, x1, y1, color, label="", lx=0.0, ly=0.0, lw=2.2):
        ax.annotate("", xy=(x1, y1), xytext=(x0, y0),
                    arrowprops=dict(arrowstyle="-|>", color=color, lw=lw,
                                    shrinkA=3, shrinkB=3, mutation_scale=18))
        if label:
            ax.text((x0 + x1) / 2 + lx, (y0 + y1) / 2 + ly, label,
                    fontsize=9.5, color=color, ha="center", va="center", weight="bold")

    node(2.2, 4.75, "光伏", "P_pv  （弃光 s）", color=C_PV, w=2.1)
    node(2.2, 1.30, "外部电网", "P_buy / 紧急购电", color=C_GRID, w=2.4)
    node(5.5, 1.30, "储能设备", "储电量 E", color=C_STO, w=2.1)
    node(8.3, 3.00, "小区负载", "P_load", color=C_LOAD, w=1.7)

    arrow(2.2, 4.16, 2.2, 3.12, C_PV, "P_pv", lx=0.46)
    arrow(2.2, 1.88, 2.2, 2.88, C_GRID, "P_buy", lx=0.52)
    arrow(6.7, 3.0, 7.44, 3.0, C_LOAD, "P_load", ly=0.44)
    arrow(5.15, 2.85, 5.15, 1.95, C_STO, "充电 P_ch", lx=-0.62, ly=0.30)
    arrow(5.85, 1.95, 5.85, 2.85, P["green"], "放电 P_dis", lx=0.66, ly=0.30)

    ax.text(5.0, 6.08, "功率平衡：  P_buy + (P_pv - s) + P_dis = P_load + P_ch",
            ha="center", fontsize=10, color="#2F3542",
            bbox=dict(boxstyle="round,pad=0.42", fc="#F7F8FA", ec="#C7CCD4", lw=0.9))

    ax.set_title("图1  微网拓扑与能量流示意", fontsize=13, weight="bold")
    fig.savefig(os.path.join(OUT, "图1_拓扑示意图.png"))
    plt.close(fig)


def fig2_price():
```

```
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(t, price, color=P["blue"], lw=2.0)
    ax.fill_between(t, price, price.min(), color=P["blue"], alpha=0.15)
    ax.set_xlabel("时间 (h)"); ax.set_ylabel("电价 (元/kWh)")
    ax.set_title("图2  分时电价曲线")
    ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
    ax.margins(x=0)
    fig.savefig(os.path.join(OUT, "图2_分时电价.png"))
    plt.close(fig)


def fig3_load_pv():
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(t, load, color=P["dark"], lw=2.0, label="小区负载")
    ax.plot(t, pv, color=P["orange"], lw=2.0, label="光伏发电预测")
    ax.fill_between(t, pv, 0, color=P["orange"], alpha=0.12)
    ax.set_xlabel("时间 (h)"); ax.set_ylabel("功率 (kW)")
    ax.set_title("图3  小区负载与光伏发电预测曲线")
    ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
    ax.margins(x=0); ax.legend(loc="upper left")
    fig.savefig(os.path.join(OUT, "图3_负载光伏.png"))
    plt.close(fig)


def fig4_balance():
    fig, ax = plt.subplots(figsize=(9, 4.5))
    ax.stackplot(t, b, pv_used, dis, labels=["购电", "光伏利用", "储能放电"],
                 colors=[P["blue"], P["orange"], P["green"]], alpha=0.9)
    ax.plot(t, load, color=P["dark"], lw=2.0, ls="--", label="小区负载")
    ax.set_xlabel("时间 (h)"); ax.set_ylabel("功率 (kW)")
    ax.set_title("图4  功率平衡堆叠面积图（电从哪来）")
    ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
    ax.margins(x=0)
    ax.legend(loc="upper left", ncol=2, frameon=True, facecolor="white", edgecolor="#E4E7EB")
    fig.savefig(os.path.join(OUT, "图4_功率平衡堆叠.png"))
    plt.close(fig)
```

```
105
106
107 def fig5_soc():
108     Efull = np.concatenate([[C.E0], E])
109     fig, ax = plt.subplots(figsize=(9, 4.5))
110     ax.plot(np.arange(C.N + 1) / 6.0, Efull, color=P["purple"], lw
    =2.2, label="储电量 (左轴)")
111     ax.axhline(C.E_LOWER, color="#C7CCD4", ls=":", lw=1.2)
112     ax.axhline(C.E_UPPER, color="#C7CCD4", ls=":", lw=1.2)
113     ax.text(0.3, C.E_UPPER + 60, "运行上限 10800", fontsize=8, color
    ="#9AA0A6")
114     ax.text(0.3, C.E_LOWER - 260, "运行下限 1200", fontsize=8, color
    ="#9AA0A6")
115     ax.set_xlabel("时间 (h)"); ax.set_ylabel("储电量 (kWh)", color=P[
    "purple"])
116     ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
117     ax.margins(x=0); ax.set_ylim(0, 12000)
118     ax2 = ax.twinx()
119     ax2.spines["right"].set_visible(True); ax2.spines["right"].
    set_color("#C7CCD4")
120     ax2.bar(t, ch, width=1/6, color=P["purple"], alpha=0.55, label=
    "充电")
121     ax2.bar(t, -dis, width=1/6, color=P["green"], alpha=0.55, label
    ="放电")
122     ax2.set_ylabel("充/放电功率 (kW)"); ax2.set_ylim(-C.P_MAX, C.
    P_MAX)
123     ax2.axhline(0, color="#C7CCD4", lw=0.8)
124     h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.
    get_legend_handles_labels()
125     ax.legend(h1 + h2, l1 + l2, loc="lower right", ncol=2, frameon=
    True, facecolor="white", edgecolor="#E4E7EB")
126     ax.set_title("图5　储能 SOC 与充放电调度（谷充峰放）")
127     fig.savefig(os.path.join(OUT, "图5_SOC充放电.png"))
128     plt.close(fig)
129
130
131 if __name__ == "__main__":
132     fig1_topo(); fig2_price(); fig3_load_pv(); fig4_balance();
    fig5_soc()
133     print("问题1 图表已生成到", OUT)
```

Listing 4　plot_q1.py（问题 1 图表生成）

B.3　问题 2 代码

B.3.1　solve ——问题 2 求解

```python
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))

price, load_fj1, pv_fj1 = C.load_fj1()
dates, load, pv = C.load_fj2()

n_days = load.shape[0]
START, END = 31, n_days

WINDOW = 5
def forecast_avg(actual, prior):
    n = actual.shape[0]
    f = np.empty_like(actual)
    for d in range(n):
        lo = max(0, d - WINDOW)
        f[d] = prior if lo == d else actual[lo:d].mean(axis=0)
    return f

load_f = forecast_avg(load, load_fj1)
pv_f   = forecast_avg(pv, pv_fj1)

# 报童临界比 c_u/(c_u+c_o)=4/5=0.8，安全裕量取预报误差的 80% 分位
net_actual = load - pv
net_f      = load_f - pv_f
err        = net_actual[WINDOW:] - net_f[WINDOW:]
q80        = np.quantile(err, 0.80, axis=0)
print(f"安全裕量 q80 均值: {q80.mean():.2f} kW（逐时段 {q80.min():.2f} ~ {q80.max():.2f}）")

```

```

def run_day(d, E_start):
    load_hedged = load_f[d] + q80
    plan = C.solve_plan_carry(price, load_hedged, pv_f[d], E_start)
    covered = net_f[d] + q80 + plan["s"]
    e = np.maximum(net_actual[d] - covered, 0.0)
    cost_plan = float(np.sum(price * plan["b"]) * C.DT)
    cost_emer = float(np.sum(5.0 * price * e) * C.DT)
    return plan, e, cost_plan, cost_emer

# 跨日连续：1月1日 0:00=6000，逐日顺延（不再日末归位）
E_carry = C.warmup_storage(lambda d: price, load, pv, START)
plan_rec, emer_rec, cost_plan_arr, cost_emer_arr = {}, {}, [], []
E_start_by_day = {}
for d in range(START, END):
    plan, e, cp, ce = run_day(d, E_carry)
    E_start_by_day[dates[d]] = E_carry
    E_carry = float(plan["E"][-1])
    plan_rec[dates[d]] = plan
    emer_rec[dates[d]] = e
    cost_plan_arr.append(cp); cost_emer_arr.append(ce)

cost_plan_arr = np.array(cost_plan_arr)
cost_emer_arr = np.array(cost_emer_arr)

print(f"问题2 结果（{str(pd.Timestamp(dates[START]).date())} ~ "
      f"{str(pd.Timestamp(dates[END-1]).date())}，共 {END-START} 天）")
print(f"全年计划购电费合计 : {cost_plan_arr.sum():,.2f} 元")
print(f"全年紧急购电费合计 : {cost_emer_arr.sum():,.2f} 元")
print(f"全年总购电费合计   : {(cost_plan_arr+cost_emer_arr).sum():,.2f} 元")
print(f"紧急购电费占比     : {cost_emer_arr.sum()/(cost_plan_arr.sum()+cost_emer_arr.sum())*100:.2f} %")
total_emer_kwh = sum(float(np.sum(e) * C.DT) for e in emer_rec.values())
print(f"全年紧急购电量合计 : {total_emer_kwh:,.2f} kWh")

spec_dates = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
date_to_idx = {str(pd.Timestamp(dates[i]).date()): i for i in range
```

```
    (n_days)}
print("\n表3  微网在指定日期的紧急购电量")
for sd in spec_dates:
    i = date_to_idx[sd]
    e = emer_rec[dates[i]]
    print(f"\n  {sd}:")
    inter = C.merge_contiguous(e)
    if not inter:
        print("    无紧急购电")
    else:
        print(f"    {'购电时间段':<18}{'购电量(kWh)':>12}")
        for (s, t, kwh) in inter:
            lbl = f"{C.period_label(s).split('-')[0]}-{C.
    period_label(t-1).split('-')[1]}"
            print(f"    {lbl:<20}{kwh:>10.4f}")
        print(f"    {'合计':<18}{float(np.sum(e)*C.DT):>12.4f}")

df_plan = {dates[d]: {"b": plan_rec[dates[d]]["b"], "price": price}
    for d in range(START, END)}
df_charge = {dates[d]: {"ch": plan_rec[dates[d]]["ch"], "dis":
    plan_rec[dates[d]]["dis"],
                        "E_start": E_start_by_day[dates[d]], "E_end
    ": plan_rec[dates[d]]["E"][-1]}
             for d in range(START, END)}
df_emer = {dates[d]: emer_rec[dates[d]] for d in range(START, END)}

out = os.path.join(HERE, "result2.xlsx")
if os.path.exists(out): os.remove(out)
C.write_plan_sheet(df_plan, out, "计划购电量")
C.write_charge_sheet(df_charge, out, "充放电量")
C.write_emergency_sheet(df_emer, out, "紧急购电量")
print(f"\n已保存 result2.xlsx 到 {HERE}")

summary = pd.DataFrame({
    "日期": [str(pd.Timestamp(dates[d]).date()) for d in range(START
    , END)],
    "计划购电费(元)": cost_plan_arr,
    "紧急购电费(元)": cost_emer_arr,
    "总购电费(元)": cost_plan_arr + cost_emer_arr,
    "紧急购电量(kWh)": [float(np.sum(emer_rec[dates[d]]) * C.DT) for
    d in range(START, END)],
```

```
})
C.write_summary(summary, os.path.join(HERE, "_q2_summary.csv"))
print("已保存汇总 _q2_summary.csv")

b_all = np.array([plan_rec[dates[d]]["b"] for d in range(START, END)])
e_all = np.array([emer_rec[dates[d]] for d in range(START, END)])
np.savez(os.path.join(HERE, "_q2_data.npz"),
         dates=dates[START:END], price=price, load=load[START:END], pv=pv[START:END],
         net_actual=net_actual[START:END], net_f=net_f[START:END],
         cost_plan=cost_plan_arr, cost_emer=cost_emer_arr,
         b=b_all, e=e_all, q80=q80)
print("已保存绘图数据 _q2_data.npz")
```

Listing 5　solve_q2.py（问题 2 两阶段报童模型求解）

B.3.2　verify ——问题 2 验证

```
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()
P = C.PALETTE

price, load_fj1, pv_fj1 = C.load_fj1()
dates, load, pv = C.load_fj2()
n_days = load.shape[0]
START, END = 31, n_days

WINDOW = 5
def forecast_avg(actual, prior):
    n = actual.shape[0]
```

```
    f = np.empty_like(actual)
    for d in range(n):
        lo = max(0, d - WINDOW)
        f[d] = prior if lo == d else actual[lo:d].mean(axis=0)
    return f

load_f = forecast_avg(load, load_fj1)
pv_f = forecast_avg(pv, pv_fj1)
net_actual = load - pv
net_f = load_f - pv_f
err = net_actual[WINDOW:] - net_f[WINDOW:]

DT = C.DT

# 跨日连续储能：仅 1月1日 0:00=6000，24:00 顺延为次日 0:00（与 solve_q2 一致）
E0_warm = C.warmup_storage(lambda d: price, load, pv, START)


def annual_cost(q):
    qq = np.quantile(err, q, axis=0)
    cp = ce = ek = 0.0
    E_carry = E0_warm
    for d in range(START, END):
        load_hedged = load_f[d] + qq
        plan = C.solve_plan_carry(price, load_hedged, pv_f[d], E_carry)
        E_carry = float(plan["E"][-1])
        covered = net_f[d] + qq + plan["s"]
        e = np.maximum(net_actual[d] - covered, 0.0)
        cp += float(np.sum(price * plan["b"]) * DT)
        ce += float(np.sum(5.0 * price * e) * DT)
        ek += float(np.sum(e) * DT)
    total = cp + ce
    return cp, ce, total, ce / total


qs = np.array([0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95])
res = np.array([annual_cost(q) for q in qs])
cp, ce, total, ratio = res[:, 0], res[:, 1], res[:, 2], res[:, 3]
```

```
q_star = qs[int(np.argmin(total))]

print("问题2 报童临界比数值验证（服务率 q 敏感性扫描，全年 334 天）")
print(f"{'服务率 q':>8} | {'计划购电费(万)':>13} | {'紧急购电费(万)':>13} | "
      f"{'总购电费(万)':>12} | {'紧急占比':>8}")
for i in range(len(qs)):
    if abs(qs[i] - q_star) < 1e-9:
        mark = " ← 数值最优"
    elif abs(qs[i] - 0.8) < 1e-9:
        mark = " ← 理论临界比 4/5"
    else:
        mark = ""
    print(f"{qs[i]:>8.2f} | {cp[i]/1e4:>13.2f} | {ce[i]/1e4:>13.2f} | "
          f"{total[i]/1e4:>12.2f} | {ratio[i]*100:>7.2f}%{mark}")
print(f"数值最优服务率 q* = {q_star:.2f}（理论临界比 4/5 = 0.80），总费用最低 {total.min()/1e4:,.2f} 万元")
print(f"q=0.80 时总费用 {total[qs == 0.80][0]/1e4:,.2f} 万元，"
      f"比数值最低仅多 {(total[qs == 0.80][0] - total.min())/1e4:.3f} 万元 "
      f"(<0.03%)，二者实质一致")
print("结论：报童临界比 4/5 是近最优的稳健选择；数值最优略偏 0.85 源于储能可部分吸收超购、"
      "实际超购成本略低于平价。")

fig, ax = plt.subplots(figsize=(9.5, 4.8))
ax.plot(qs, total / 1e4, color=P["dark"], lw=2.4, marker="o", ms=5,
        label="全年总购电费")
ax.plot(qs, cp / 1e4, color=P["blue"], lw=1.8, marker="s", ms=4,
        label="计划购电费")
ax.plot(qs, ce / 1e4, color=P["red"], lw=1.8, marker="^", ms=4,
        label="紧急购电费")
ax.axvline(0.80, color=P["gray"], ls="--", lw=1.4)
ax.axvline(q_star, color=P["teal"], ls=":", lw=1.4)
ax.annotate("理论 q=0.80(临界比 4/5)\n数值最优 q≈0.85(相差<0.03%)", xy=(0.80, total[qs == 0.80][0] / 1e4),
            xytext=(0.62, total.max() / 1e4 * 0.92),
            arrowprops=dict(arrowstyle="->", color="#6B7280", lw=1.2),
            fontsize=9, color="#2F3542")
```

```
ax.set_xlabel("服务率 q（计划覆盖净负荷的分位）")
ax.set_ylabel("全年购电费（万元）")
ax.set_xlim(0.48, 0.97); ax.set_xticks(qs)
ax.set_title("验证图　报童临界比最优性：全年费用随服务率 q 的变化")
ax.legend(loc="lower left", bbox_to_anchor=(0.02, 0.02), frameon=
    True, facecolor="white", edgecolor="#E4E7EB")
ax.grid(axis="x", visible=False)
fig.tight_layout()
fig.savefig(os.path.join(OUT, "验证图_报童临界比最优性.png"))
plt.close(fig)

print(f"\n验证图已生成到 {OUT}")
```

Listing 6　verify_q2.py（问题 2 临界比敏感性验证）

B.3.3　plot ——问题 2 绘图

```
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
    __file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()

d = np.load(os.path.join(HERE, "_q2_data.npz"))
dates = d["dates"]; price = d["price"]; load = d["load"]; pv = d["
    pv"]
net_actual = d["net_actual"]; net_f = d["net_f"]
cost_plan = d["cost_plan"]; cost_emer = d["cost_emer"]
b = d["b"]; e = d["e"]; q80 = d["q80"]

n_days = len(dates)
day_idx = np.arange(n_days)
```

```
date_lbl = [str(pd.Timestamp(dates[i]).date())[5:] for i in range(n_days)]
t = np.arange(C.N) / 6.0
P = C.PALETTE


def fig6_flow():
    fig, ax = plt.subplots(figsize=(9, 4.6))
    ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 6)
    box1 = dict(boxstyle="round,pad=0.5", fc="#EDF3FA", ec=P["blue"], lw=1.5)
    box2 = dict(boxstyle="round,pad=0.5", fc="#FBEEEC", ec=P["red"], lw=1.5)
    ax.text(2.35, 4.6, "第一阶段（0:00 制定，一次性承诺）\n报童型计划：预报净负荷 + 80% 分位安全裕量",
            ha="center", va="center", bbox=box1, fontsize=10.5, weight="bold", color=P["blue"])
    ax.text(2.35, 3.0, "计划购电 b_plan\n储能充放电 ch / dis", ha="center", va="center",
            bbox=box1, fontsize=10.5)
    ax.text(7.65, 4.6, "第二阶段（实时）\n实际负载 / 光伏实现", ha="center", va="center",
            bbox=box2, fontsize=10.5, weight="bold", color=P["red"])
    ax.text(7.65, 3.0, "实时缺口 e = max(0, 实际净负荷 - 计划覆盖)\n按 5 倍电价紧急购电补偿", ha="center",
            va="center", bbox=box2, fontsize=10)
    ax.annotate("", xy=(6.05, 3.9), xytext=(4.0, 3.9),
                arrowprops=dict(arrowstyle="-|>", color="#6B7280", lw=2))
    ax.text(5.0, 4.08, "偏差", fontsize=9, color="#6B7280", ha="center")
    ax.text(5.0, 1.15, "报童临界比：c_u/(c_u+c_o) = 4p/5p = 0.8  →  计划覆盖 80% 分位",
            ha="center", va="center", fontsize=9.5, color="#2F3542",
            bbox=dict(boxstyle="round,pad=0.35", fc="#F7F8FA", ec="#C7CCD4", lw=0.9))
    ax.set_title("图6　问题2 两阶段「计划—补偿」流程示意", fontsize=12, weight="bold")
    fig.savefig(os.path.join(OUT, "图6_两阶段流程示意.png"))
```

```
    plt.close(fig)


def fig7_forecast():
    pick = [150, 300, 60]
    fig, axs = plt.subplots(3, 1, figsize=(9.5, 10), sharex=True)
    for k, di in enumerate(pick):
        ax = axs[k]
        ax.plot(t, net_f[di], color=P["blue"], lw=1.7, label="预报净负荷（前5日平均）")
        ax.plot(t, net_actual[di], color=P["dark"], lw=1.7, label="实际净负荷")
        ax.fill_between(t, net_f[di], net_actual[di], where=net_actual[di] > net_f[di],
                        color=P["red"], alpha=0.20, label="预报偏低→缺口")
        ax.set_ylabel("净负荷 (kW)")
        ax.set_title(f"  {date_lbl[di]}  前5日平均预报 vs 实际净负荷", fontsize=10.5, loc="left")
        ax.margins(x=0)
    axs[-1].set_xlabel("时间 (h)")
    axs[-1].set_xticks(np.arange(0, 25, 3))
    fig.suptitle("图7  前5日平均预报与实际的净负荷偏差（净负荷 = 负载 - 光伏）",
                 fontsize=13, weight="bold", y=0.99)
    fig.tight_layout(rect=[0, 0, 1, 0.96])
    fig.savefig(os.path.join(OUT, "图7_预测vs实际.png"))
    plt.close(fig)


def fig8_decompose():
    fig, axs = plt.subplots(2, 1, figsize=(10, 8), gridspec_kw={"height_ratios": [2, 1]})

    di = 150
    ax = axs[0]
    ax.stackplot(t, b[di], e[di], labels=["计划购电", "紧急购电"],
                 colors=[P["blue"], P["red"]], alpha=0.88)
    ax.plot(t, net_actual[di], color=P["dark"], lw=1.6, ls="--", label="实际净负荷")
    ax.set_ylabel("功率 (kW)"); ax.set_xlim(0, 24)
```

```
    ax.set_title(f"  {date_lbl[di]}  计划购电与紧急购电分解", fontsize=10.5, loc="left")
    ax.legend(loc="upper left", ncol=3, fontsize=9, frameon=True, facecolor="white", edgecolor="#E4E7EB")
    ax.margins(x=0)

    b_kwh = np.sum(b, axis=1) * C.DT
    e_kwh = np.sum(e, axis=1) * C.DT
    ax = axs[1]
    ax.bar(day_idx, b_kwh, width=0.85, color=P["blue"], label="计划购电量")
    ax.bar(day_idx, e_kwh, width=0.85, bottom=b_kwh, color=P["red"], label="紧急购电量")
    ax.set_xlabel("日期"); ax.set_ylabel("购电量 (kWh)")
    ax.set_title("  全年逐日计划 / 紧急购电量（堆叠）", fontsize=10.5, loc="left")
    ax.legend(loc="upper left", fontsize=9)
    ax.set_xlim(-0.5, n_days - 0.5)
    ax.set_xticks(np.arange(0, n_days, 30))
    ax.set_xticklabels([date_lbl[i] for i in range(0, n_days, 30)], fontsize=8)
    ax.grid(axis="x", visible=False)

    fig.suptitle("图8  计划购电与紧急购电的分解", fontsize=13, weight="bold", y=0.99)
    fig.tight_layout(rect=[0, 0, 1, 0.96])
    fig.savefig(os.path.join(OUT, "图8_计划vs紧急堆叠.png"))
    plt.close(fig)


def fig9_heatmap():
    fig, ax = plt.subplots(figsize=(10, 7))
    emer_kwh = e * C.DT
    im = ax.imshow(emer_kwh.T, aspect="auto", origin="lower", cmap="YlOrRd",
                   extent=[0, n_days, 0, 24], vmin=0)
    ax.grid(False)
    ax.set_xlabel("日期"); ax.set_ylabel("时间 (h)")
    ax.set_title("图9  全年紧急购电热力图（颜色 = 紧急购电量 kWh）", fontsize=12, weight="bold")
    ax.set_xticks(np.arange(0, n_days, 30))
```

```
    ax.set_xticklabels([date_lbl[i] for i in range(0, n_days, 30)],
    fontsize=8)
    ax.set_yticks(np.arange(0, 25, 3))
    cb = fig.colorbar(im, ax=ax, pad=0.015, fraction=0.046)
    cb.set_label("紧急购电量 (kWh)")
    cb.outline.set_visible(False)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "图9_紧急热力图.png"))
    plt.close(fig)


def fig10_cost():
    fig, ax = plt.subplots(figsize=(10, 5))
    ax.plot(day_idx, cost_plan, color=P["blue"], lw=1.5, label="计划
    购电费")
    ax.plot(day_idx, cost_emer, color=P["red"], lw=1.5, label="紧急
    购电费")
    ax.plot(day_idx, cost_plan + cost_emer, color=P["dark"], lw
    =2.0, label="总购电费")
    ax.set_xlabel("日期"); ax.set_ylabel("购电费 (元)")
    ax.set_title("图10  全年逐日购电费趋势")
    ax.set_xlim(-0.5, n_days - 0.5)
    ax.set_xticks(np.arange(0, n_days, 30))
    ax.set_xticklabels([date_lbl[i] for i in range(0, n_days, 30)],
    fontsize=8)
    ax.legend(loc="upper left")
    ax.grid(axis="x", visible=False)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "图10_全年费用趋势.png"))
    plt.close(fig)


if __name__ == "__main__":
    fig6_flow(); fig7_forecast(); fig8_decompose(); fig9_heatmap();
    fig10_cost()
    print("问题2 图表已生成到", OUT)
```

Listing 7　plot_q2.py（问题 2 图表生成）

B.4　问题 3 代码

B.4.1　solve ——问题 3 求解

```python
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
    __file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))

price, _, _ = C.load_fj1()
dates, load, pv = C.load_fj2()
fc_dates, fc = C.load_fj3()

n_days = load.shape[0]
START, END = 31, n_days


def run_day(d, E_start):
    load_d = load[d]; pv_actual = pv[d]
    pv_f = [None] * 4
    pv_f[0] = C.forecast_to_10min(fc[d, 0, :], 0, 0.0)
    pv_f[1] = C.forecast_to_10min(fc[d, 1, :], 6, pv_f[0][36])
    pv_f[2] = C.forecast_to_10min(fc[d, 2, :], 12, pv_f[1][72])
    pv_f[3] = C.forecast_to_10min(fc[d, 3, :], 18, pv_f[2][108])

    b_final = np.zeros(C.N); ch_final = np.zeros(C.N)
    dis_final = np.zeros(C.N); E_final = np.zeros(C.N)
    E_cur = E_start
    b0 = dis0 = ch0 = None
    for k in range(4):
        t0 = k * 36
        res = C.solve_plan_carry(price[t0:], load_d[t0:], pv_f[k][
    t0:], E_cur)
        b_final[t0:t0 + 36]   = res["b"][:36]
        ch_final[t0:t0 + 36]  = res["ch"][:36]
```

```
        dis_final[t0:t0 + 36] = res["dis"][:36]
        E_final[t0:t0 + 36]   = res["E"][:36]
        E_cur = res["E"][35]
        if k == 0:
            b0, dis0, ch0 = res["b"], res["dis"], res["ch"]
    return b0, b_final, ch_final, dis_final, E_final, ch0, dis0


def settle(b_plan, b_adj, load_d, pv_actual, dis_f, ch_f):
    normal  = np.minimum(b_plan, b_adj)
    deficit = np.maximum(b_plan - b_adj, 0.0)
    excess  = np.maximum(b_adj - b_plan, 0.0)
    cost_plan = float(np.sum(normal * price) * C.DT)
    cost_adj  = float(np.sum((deficit * 0.5 + excess * 1.5) * price) * C.DT)
    net_actual = load_d - pv_actual
    covered = b_adj + dis_f - ch_f
    e = np.maximum(net_actual - covered, 0.0)
    cost_emer = float(np.sum(5.0 * price * e) * C.DT)
    return cost_plan, cost_adj, cost_emer, e


plan_rec, adj_rec, chg_rec, emer_rec = {}, {}, {}, {}
cost_plan_arr, cost_adj_arr, cost_emer_arr = [], [], []
cost_A_arr = []
E_carry = C.warmup_storage(lambda d: price, load, pv, START)
for d in range(START, END):
    E0_day = E_carry
    b0, b_adj, ch_f, dis_f, E_f, ch0, dis0 = run_day(d, E_carry)
    E_carry = float(E_f[-1])
    cp, ca, ce, e = settle(b0, b_adj, load[d], pv[d], dis_f, ch_f)
    plan_rec[dates[d]] = b0
    adj_rec[dates[d]] = b_adj
    chg_rec[dates[d]] = {"ch": ch_f, "dis": dis_f, "E_start": E0_day, "E_end": E_f[-1]}
    emer_rec[dates[d]] = e
    cost_plan_arr.append(cp); cost_adj_arr.append(ca); cost_emer_arr.append(ce)
    net_actual = load[d] - pv[d]
    covA = b0 + dis0 - ch0
    eA = np.maximum(net_actual - covA, 0.0)
```

```
    cost_A_arr.append(float(np.sum(price * b0) * C.DT + np.sum(5.0 * price * eA) * C.DT))

cost_plan_arr = np.array(cost_plan_arr)
cost_adj_arr = np.array(cost_adj_arr)
cost_emer_arr = np.array(cost_emer_arr)
cost_A_arr = np.array(cost_A_arr)
total_arr = cost_plan_arr + cost_adj_arr + cost_emer_arr

print(f"问题3 结果（{str(pd.Timestamp(dates[START]).date())} ~ "
      f"{str(pd.Timestamp(dates[END-1]).date())}，共 {END-START} 天）")
print(f"全年计划购电费合计 : {cost_plan_arr.sum():,.2f} 元")
print(f"全年调整相关费合计 : {cost_adj_arr.sum():,.2f} 元（违约+超额）")
print(f"全年紧急购电费合计 : {cost_emer_arr.sum():,.2f} 元")
print(f"全年总购电费合计   : {total_arr.sum():,.2f} 元")
print(f"  └ 紧急占比 {cost_emer_arr.sum()/total_arr.sum()*100:.2f}% | "
      f"调整占比 {cost_adj_arr.sum()/total_arr.sum()*100:.2f}%")
print(f"\n[预报价值分析] 0点-only（不滚动）总购电费 : {cost_A_arr.sum():,.2f} 元")
print(f"  滚动更新带来的全年节省 : {cost_A_arr.sum()-total_arr.sum():,.2f} 元 "
      f"({(cost_A_arr.sum()-total_arr.sum())/cost_A_arr.sum()*100:.2f}%)")

spec_dates = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
date_to_idx = {str(pd.Timestamp(dates[i]).date()): i for i in range(n_days)}
print("\n表3  微网在指定日期的紧急购电量")
for sd in spec_dates:
    i = date_to_idx[sd]
    e = emer_rec[dates[i]]
    print(f"\n  {sd}:")
    inter = C.merge_contiguous(e)
    if not inter:
        print("    无紧急购电")
    else:
        print(f"    {'购电时间段':<18}{'购电量(kWh)':>12}")
        for (s, t, kwh) in inter:
```

```
            lbl = f"{C.period_label(s).split('-')[0]}-{C.period_label(t-1).split('-')[1]}"
            print(f"    {lbl:<20}{kwh:>10.4f}")
        print(f"    {'合计':<18}{float(np.sum(e)*C.DT):>12.4f}")

out = os.path.join(HERE, "result3.xlsx")
if os.path.exists(out): os.remove(out)

def write_purchase_sheet(rec, sheet, fee_map):
    cols = ["日期\\时间"] + C.period_labels_all() + ["全天购电量", "全天购电费"]
    rows = []
    for d in range(START, END):
        dt = dates[d]
        kwh = np.round(rec[dt] * C.DT, 4)
        rows.append([C._fmt_date(dt)] + list(kwh) + [round(float(np.sum(kwh)), 4),
                                                     round(float(fee_map[dt]), 2)])
    C._append_sheet(pd.DataFrame(rows, columns=cols), out, sheet)

fee_plan = {dates[d]: cost_plan_arr[d - START] for d in range(START, END)}
write_purchase_sheet(plan_rec, "计划购电量", fee_plan)
fee_adj = {dates[d]: cost_adj_arr[d - START] for d in range(START, END)}
write_purchase_sheet(adj_rec, "调整购电量", fee_adj)

chg_rows = []
for d in range(START, END):
    rec = chg_rec[dates[d]]
    ch4, dis4 = C.aggregate_4h(rec["ch"]), C.aggregate_4h(rec["dis"])
    date_str = C._fmt_date(dates[d])
    for p in range(6):
        tc = "0:00" if p == 0 else ("24:00" if p == 1 else "")
        ec = round(rec["E_start"], 1) if p == 0 else (round(rec["E_end"], 1) if p == 1 else "")
        chg_rows.append([date_str if p == 0 else "", C.FOUR_H_LABELS[p],
                         round(ch4[p], 2), round(dis4[p], 2), tc,
```

```
    ec])
C._append_sheet(pd.DataFrame(chg_rows, columns=["日期", "时间段", "充电量", "放电量", "时刻", "储电量"]),
                out, "充放电量")

emer_rows = []
for d in range(START, END):
    e = emer_rec[dates[d]]
    date_str = C._fmt_date(dates[d])
    inter = C.merge_contiguous(e)
    if not inter:
        emer_rows.append([date_str, "", ""])
    else:
        for (s, t, kwh) in inter:
            lbl = f"{C.period_label(s).split('-')[0]}-{C.period_label(t-1).split('-')[1]}"
            emer_rows.append([date_str, lbl, round(kwh, 4)])
C._append_sheet(pd.DataFrame(emer_rows, columns=["日期", "购电时间段", "购电量"]),
                out, "紧急购电量")
print(f"\n已保存 result3.xlsx 到 {HERE}")

summary = pd.DataFrame({
    "日期": [str(pd.Timestamp(dates[d]).date()) for d in range(START, END)],
    "计划购电费(元)": cost_plan_arr,
    "调整相关费(元)": cost_adj_arr,
    "紧急购电费(元)": cost_emer_arr,
    "总购电费(元)": total_arr,
    "0点only总费用(元)": cost_A_arr,
    "紧急购电量(kWh)": [float(np.sum(emer_rec[dates[d]]) * C.DT) for d in range(START, END)],
})
C.write_summary(summary, os.path.join(HERE, "_q3_summary.csv"))
print("已保存汇总 _q3_summary.csv")

b_plan_all = np.array([plan_rec[dates[d]] for d in range(START, END)])
b_adj_all = np.array([adj_rec[dates[d]] for d in range(START, END)])
e_all = np.array([emer_rec[dates[d]] for d in range(START, END)])
```

```
ch_all = np.array([chg_rec[dates[d]]["ch"] for d in range(START, END)])
dis_all = np.array([chg_rec[dates[d]]["dis"] for d in range(START, END)])
np.savez(os.path.join(HERE, "_q3_data.npz"),
         dates=dates[START:END], price=price, load=load[START:END], pv=pv[START:END],
         b_plan=b_plan_all, b_adj=b_adj_all, e=e_all, ch=ch_all, dis=dis_all,
         cost_plan=cost_plan_arr, cost_adj=cost_adj_arr, cost_emer=cost_emer_arr,
         cost_A=cost_A_arr)
print("已保存绘图数据 _q3_data.npz")
```

Listing 8　solve_q3.py（问题 3 滚动 MPC 求解）

B.4.2　verify ——问题 3 验证

```
# -*- coding: utf-8 -*-
"""
问题3 验证：预报更新次数敏感性 —— 是否需要在 0/6/12/18 四次预报之后再引入更多预报?
思路：
  1) 用真实预报（附件3 的 0/6/12/18 四条曲线）构造 1/2/3/4 次更新的滚动调度，看费用如何随更新次数下降；
  2) 给出"完美预报下界"（用附件2 实际光伏直接求解）作为理论下限；
  3) 对 >4 次的"更多预报"，由于附件3 只有 4 条真实曲线，中间时点只能线性插值（不新增信息量），
     观察其费用变化，从而判断"更多预报"是否值得引入。
"""
import os
import sys
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()
```

```
P = C.PALETTE

price, _, _ = C.load_fj1()
dates, load, pv = C.load_fj2()
_, fc = C.load_fj3()
n_days = load.shape[0]
START, END = 31, n_days
DT, N = C.DT, C.N


def real_forecast_curves(d):
    """附件3 当天 4 条真实预报（含时段衔接锚点），与 solve_q3 完全一致。"""
    pv_f = [None] * 4
    pv_f[0] = C.forecast_to_10min(fc[d, 0, :], 0, 0.0)
    pv_f[1] = C.forecast_to_10min(fc[d, 1, :], 6, pv_f[0][36])
    pv_f[2] = C.forecast_to_10min(fc[d, 2, :], 12, pv_f[1][72])
    pv_f[3] = C.forecast_to_10min(fc[d, 3, :], 18, pv_f[2][108])
    return pv_f


def forecast_at(pv_f, h):
    """任意时刻 h 发布的预报：真实预报时点取原值，中间时点线性插值。"""
    if h <= 0:
        return pv_f[0]
    if h >= 18:
        return pv_f[3]
    k = int(h // 6)
    a = (h - 6 * k) / 6.0
    return (1 - a) * pv_f[k] + a * pv_f[k + 1]


def run_day_freq(d, hours, E_start):
    """按给定更新时刻列表 hours（小时）滚动求解，返回 (0点计划 b0, 最终购电 b, ch, dis, 日末储电量)。
    储能跨日连续：只给定 1月1日 0:00=6000，24:00 储电量顺延为次日 0:00（不再日末归位）。"""
    load_d = load[d]
    pv_f = real_forecast_curves(d)
    b = np.zeros(N); ch = np.zeros(N); dis = np.zeros(N)
    E_cur = E_start
    b0 = None
```

```
    m = len(hours)
    for i in range(m):
        t0 = int(round(hours[i] * 6))
        t1 = int(round(hours[i + 1] * 6)) if i + 1 < m else N
        fc_cur = forecast_at(pv_f, hours[i])
        res = C.solve_plan_carry(price[t0:], load_d[t0:], fc_cur[t0:], E_cur)
        b[t0:t1]   = res["b"][:t1 - t0]
        ch[t0:t1]  = res["ch"][:t1 - t0]
        dis[t0:t1] = res["dis"][:t1 - t0]
        E_cur = res["E"][t1 - t0 - 1]
        if i == 0:
            b0 = res["b"]
    return b0, b, ch, dis, float(E_cur)


def settle(b0, b_f, load_d, pv_actual, dis_f, ch_f):
    """与 solve_q3 相同的结算：计划 100% / 违约 50% / 超额 150% / 实时缺口 5 倍。"""
    normal  = np.minimum(b0, b_f)
    deficit = np.maximum(b0 - b_f, 0.0)
    excess  = np.maximum(b_f - b0, 0.0)
    cost_plan = float(np.sum(normal * price) * DT)
    cost_adj  = float(np.sum((deficit * 0.5 + excess * 1.5) * price) * DT)
    covered = b_f + dis_f - ch_f
    e = np.maximum((load_d - pv_actual) - covered, 0.0)
    cost_emer = float(np.sum(5.0 * price * e) * DT)
    return cost_plan + cost_adj + cost_emer


def annual_cost(hours):
    tot = 0.0
    E_carry = C.warmup_storage(lambda d: price, load, pv, START)
    for d in range(START, END):
        b0, b_f, ch_f, dis_f, E_end = run_day_freq(d, hours, E_carry)
        E_carry = E_end
        tot += settle(b0, b_f, load[d], pv[d], dis_f, ch_f)
    return tot

```

```python

def perfect_floor():
    """完美预报下界：用实际光伏（负荷已知）一次性求解的确定性最优（跨日连续）。
    """
    tot = 0.0
    E_carry = C.warmup_storage(lambda d: price, load, pv, START)
    for d in range(START, END):
        res = C.solve_plan_carry(price, load[d], pv[d], E_carry)
        E_carry = float(res["E"][-1])
        tot += float(np.sum(price * res["b"]) * DT)
    return tot


# ---- 真实预报：1/2/3/4 次更新 ----
real_scheds = {
    1: [0.0],
    2: [0.0, 12.0],
    3: [0.0, 6.0, 12.0],
    4: [0.0, 6.0, 12.0, 18.0],
}
real_costs = {}
print("问题3 验证：预报更新次数敏感性（全年 334 天）")
print("【真实预报】")
print(f"{'更新次数':>6} | {'更新时刻':<20} | {'全年总购电费(万元)':>16} | {'相对1次节省':>10}")
base = None
for F in [1, 2, 3, 4]:
    hours = real_scheds[F]
    c = annual_cost(hours)
    real_costs[F] = c
    if F == 1:
        base = c
    hl = "+".join(f"{int(h)}" for h in hours)
    print(f"{F:>6} | {hl:<20} | {c/1e4:>16.2f} | {(base-c)/base*100:>9.2f}%")

floor = perfect_floor()
print(f"\n【完美预报下界】{floor/1e4:>16.2f} 万元（用附件2 实际光伏，等于无限次预报的极限）")

# ---- 更多预报（>4 次）：中间时点线性插值，不新增信息 ----
```

```
print("\n【更多预报（中间时点线性插值，不新增信息）】")
print(f"{'更新次数':>6} | {'全年总购电费(万元)':>16} | {'相对1次节省':>10}")
interp = {}
for F in [6, 12, 24]:
    hours = np.linspace(0, 24, F + 1)[:-1]
    c = annual_cost(hours)
    interp[F] = c
    print(f"{F:>6} | {c/1e4:>16.2f} | {(base-c)/base*100:>9.2f}%")

print("\n结论：真实预报从 1 次加到 4 次，费用单调下降且边际递减（1→2 省约"
      f"{(real_costs[1]-real_costs[2])/1e4:.1f} 万，2→4 省约{(real_costs[2]-real_costs[4])/1e4:.1f} 万）；")
print("      4 次之后继续加密只能插值、不新增信息，费用不降反升，故无需引入更多预报。")

# ---- 折线图：真实预报 + 完美下界 ----
F_real = [1, 2, 3, 4]
c_real = [real_costs[F] / 1e4 for F in F_real]

fig, ax = plt.subplots(figsize=(8.5, 5.2))
ax.plot(F_real, c_real, color=P["blue"], lw=2.2, marker="o", ms=7,
        label="滚动 MPC（真实预报）")
ax.axhline(floor / 1e4, color=P["red"], lw=1.6, ls="--",
           label=f"完美预报下界 {floor/1e4:,.0f} 万元")
# 标注当前方案与收敛
ax.annotate("当前方案：4 次（0/6/12/18）", xy=(4, c_real[-1]),
            xytext=(2.25, c_real[-1] + 45),
            arrowprops=dict(arrowstyle="->", color="#6B7280", lw=1.2),
            fontsize=9.5, color="#2F3542", weight="bold")
ax.annotate("继续加密只能插值\n（不新增信息，费用反升）", xy=(3.55, c_real[-1] + 10),
            xytext=(1.15, floor/1e4 + 18),
            arrowprops=dict(arrowstyle="->", color=P["red"], lw=1.1),
            fontsize=9, color=P["red"])

ax.set_xticks(F_real)
ax.set_xticklabels([str(f) for f in F_real])
ax.set_xlabel("每日预报更新次数")
```

```
ax.set_ylabel("全年总购电费（万元）")
ax.set_xlim(0.7, 4.3)
ax.set_ylim(floor/1e4 - 40, c_real[0] + 80)
ax.set_title("验证图　预报更新次数敏感性：4 次真实预报后费用收敛")
ax.grid(axis="x", visible=False)
ax.grid(axis="y", alpha=0.3)
ax.legend(loc="upper right", frameon=True, facecolor="white",
    edgecolor="#E4E7EB")
fig.tight_layout()
fig.savefig(os.path.join(OUT, "验证图_预报更新频率敏感性.png"))
plt.close(fig)

print(f"\n验证图已生成到 {OUT}")
```

Listing 9　verify_q3.py（问题 3 滚动频率敏感性验证）

B.4.3　plot ——问题 3 绘图

```
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
    __file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()

d = np.load(os.path.join(HERE, "_q3_data.npz"))
dates = d["dates"]; price = d["price"]; load = d["load"]; pv = d["
    pv"]
b_plan = d["b_plan"]; b_adj = d["b_adj"]; e = d["e"]; ch = d["ch"];
     dis = d["dis"]
cost_plan = d["cost_plan"]; cost_adj = d["cost_adj"]; cost_emer = d
    ["cost_emer"]
cost_A = d["cost_A"]

```

```
n_days = len(dates)
day_idx = np.arange(n_days)
date_lbl = [str(pd.Timestamp(dates[i]).date())[5:] for i in range(n_days)]
t = np.arange(C.N) / 6.0
total = cost_plan + cost_adj + cost_emer
P = C.PALETTE


def fig11_mpc():
    fig, ax = plt.subplots(figsize=(9.5, 4.4))
    ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 5)
    ax.plot([1, 9], [0.6, 0.6], color="#C7CCD4", lw=1.5)
    ax.text(5, 0.26, "时间轴（0:00 ~ 24:00）", ha="center", fontsize=9, color="#6B7280")
    nodes = [("0:00\n制定全天计划", 1.6, P["blue"]), ("6:00\n更新预报·调整", 3.8, P["orange"]),
             ("12:00\n更新预报·调整", 6.0, P["orange"]), ("18:00\n更新预报·调整", 8.2, P["orange"])]
    for lab, x, c in nodes:
        ax.plot([x, x], [0.6, 0.95], color=c, lw=1.6)
        ax.scatter([x], [0.62], s=50, color=c, zorder=3)
        ax.text(x, 1.08, lab, ha="center", va="bottom", fontsize=9.5, color=c, weight="bold")
    for x, w, c in [(1.6, 3.2, P["blue"]), (3.8, 3.2, P["orange"]),
                    (6.0, 3.2, P["orange"]), (8.2, 3.2, P["orange"])]:
        ax.add_patch(plt.Rectangle((x - w / 2, 1.65), w, 1.55, fill=True, fc=c, alpha=0.10,
                                   ec=c, lw=1.2, ls="--"))
    ax.text(5, 3.95, "每个时刻用最新 24h 预报重新优化剩余时段（储能状态顺延）",
            ha="center", fontsize=10.5, color="#2F3542")
    ax.text(5, 4.55, "结算：计划 vs 调整 —— 违约(50%) / 超额(150%)，实时缺口 5 倍紧急购电",
            ha="center", fontsize=9.5, color=P["red"], weight="bold")
    ax.set_title("图11　问题3 滚动 MPC 原理示意", fontsize=12, weight="bold")
    fig.savefig(os.path.join(OUT, "图11_MPC原理示意.png"))
    plt.close(fig)
```

```

def fig12_forecast():
    _, fc = C.load_fj3()
    di = 150
    di_full = 31 + di
    pv0 = C.forecast_to_10min(fc[di_full, 0, :], 0, 0.0)
    pv6 = C.forecast_to_10min(fc[di_full, 1, :], 6, pv0[36])
    pv12 = C.forecast_to_10min(fc[di_full, 2, :], 12, pv6[72])
    pv18 = C.forecast_to_10min(fc[di_full, 3, :], 18, pv12[108])
    fig, ax = plt.subplots(figsize=(9.5, 4.6))
    ax.plot(t, pv[di], color=P["dark"], lw=2.2, label="实际光伏（附件2）")
    ax.plot(t, pv0, color=P["blue"], lw=1.4, ls="--", label="0:00预报")
    ax.plot(t, pv6, color=P["orange"], lw=1.4, ls="--", label="6:00预报")
    ax.plot(t, pv12, color=P["purple"], lw=1.4, ls="--", label="12:00 预报")
    ax.plot(t, pv18, color=P["teal"], lw=1.4, ls="--", label="18:00预报")
    for h in [6, 12, 18]:
        ax.axvline(h, color="#C7CCD4", ls=":", lw=1.2)
        ax.text(h + 0.15, ax.get_ylim()[1] * 0.95, f"{h}:00", fontsize=8, color="#9AA0A6")
    ax.set_xlabel("时间 (h)"); ax.set_ylabel("光伏功率 (kW)")
    ax.set_title(f"图12　光伏预报逐次更新收敛（{date_lbl[di]}，竖线为更新时刻）")
    ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
    ax.margins(x=0)
    ax.legend(loc="upper left", ncol=2, frameon=True, facecolor="white", edgecolor="#E4E7EB")
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "图12_预报更新对比.png"))
    plt.close(fig)


def fig13_decompose():
    fig, axs = plt.subplots(2, 1, figsize=(10, 8), gridspec_kw={"height_ratios": [2, 1]})
    di = 150
```

```
    ax = axs[0]
    ax.stackplot(t, b_plan[di], b_adj[di] - b_plan[di], e[di],
                 labels=["计划购电", "调整购电增量", "紧急购电"],
                 colors=[P["blue"], P["orange"], P["red"]], alpha=0.88)
    ax.plot(t, load[di] - pv[di], color=P["dark"], lw=1.6, ls="--", label="实际净负荷")
    ax.set_ylabel("功率 (kW)"); ax.set_xlim(0, 24)
    ax.set_title(f"  {date_lbl[di]}  计划 / 调整 / 紧急购电分解", fontsize=10.5, loc="left")
    ax.legend(loc="upper left", ncol=4, fontsize=8.5, frameon=True, facecolor="white", edgecolor="#E4E7EB")
    ax.margins(x=0)

    ax = axs[1]
    ax.bar(day_idx, cost_plan, width=0.85, color=P["blue"], label="计划购电费")
    ax.bar(day_idx, cost_adj, width=0.85, bottom=cost_plan, color=P["orange"], label="调整相关费")
    ax.bar(day_idx, cost_emer, width=0.85, bottom=cost_plan + cost_adj,
           color=P["red"], label="紧急购电费")
    ax.set_xlabel("日期"); ax.set_ylabel("购电费 (元)")
    ax.set_title("  全年逐日费用分解（堆叠）", fontsize=10.5, loc="left")
    ax.legend(loc="upper left", fontsize=9)
    ax.set_xlim(-0.5, n_days - 0.5)
    ax.set_xticks(np.arange(0, n_days, 30))
    ax.set_xticklabels([date_lbl[i] for i in range(0, n_days, 30)], fontsize=8)
    ax.grid(axis="x", visible=False)

    fig.suptitle("图13  计划 / 调整 / 紧急购电的费用分解", fontsize=13, weight="bold", y=0.99)
    fig.tight_layout(rect=[0, 0, 1, 0.96])
    fig.savefig(os.path.join(OUT, "图13_计划调整紧急分解.png"))
    plt.close(fig)


def fig14_value():
    fig, axs = plt.subplots(2, 1, figsize=(10, 7), gridspec_kw={"
```

```
    height_ratios": [1, 1]})
114     save = cost_A - total
115     ax = axs[0]
116     ax.plot(day_idx, cost_A, color=P["gray"], lw=1.5, label="0点-
    only（不滚动）")
117     ax.plot(day_idx, total, color=P["blue"], lw=2.0, label="滚动 MPC
    ")
118     ax.set_ylabel("逐日购电费（元）")
119     ax.set_title("  逐日购电费对比", fontsize=10.5, loc="left")
120     ax.legend(loc="upper left")
121     ax.set_xlim(-0.5, n_days - 0.5)
122     ax.set_xticks(np.arange(0, n_days, 30)); ax.set_xticklabels([])
123     ax.grid(axis="x", visible=False)
124
125     ax = axs[1]
126     ax.plot(day_idx, save, color=P["orange"], lw=1.4, label="逐日节
    省")
127     ax.plot(day_idx, np.cumsum(save), color=P["teal"], lw=2.0,
    label="累计节省")
128     ax.axhline(0, color="#C7CCD4", lw=1.0)
129     ax.fill_between(day_idx, 0, save, where=save > 0, color=P["
    orange"], alpha=0.15)
130     ax.set_xlabel("日期"); ax.set_ylabel("节省费用（元）")
131     ax.set_title("  滚动更新节省费用（累计 = 预报价值）", fontsize=10.5,
    loc="left")
132     ax.legend(loc="upper left")
133     ax.set_xlim(-0.5, n_days - 0.5)
134     ax.set_xticks(np.arange(0, n_days, 30))
135     ax.set_xticklabels([date_lbl[i] for i in range(0, n_days, 30)],
    fontsize=8)
136     ax.grid(axis="x", visible=False)
137
138     fig.suptitle(f"图14  滚动预报更新的价值（全年累计节省 {save.sum():,.0
    f} 元）",
139                  fontsize=13, weight="bold", y=0.99)
140     fig.tight_layout(rect=[0, 0, 1, 0.96])
141     fig.savefig(os.path.join(OUT, "图14_预报价值分析.png"))
142     plt.close(fig)
143
144
145 if __name__ == "__main__":
```

```
    fig11_mpc(); fig12_forecast(); fig13_decompose(); fig14_value()
    print("问题3 图表已生成到", OUT)
```

Listing 10　plot_q3.py（问题 3 图表生成）

B.5　问题 4 代码

B.5.1　solve ——问题 4 求解

```
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
    __file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))

_, load_fj1, pv_fj1 = C.load_fj1()
dates, load, pv = C.load_fj2()
_, fc = C.load_fj3()
_, price4 = C.load_fj4()

n_days = load.shape[0]
START, END = 31, n_days

WINDOW = 5
def forecast_avg(actual, prior):
    n = actual.shape[0]
    f = np.empty_like(actual)
    for d in range(n):
        lo = max(0, d - WINDOW)
        f[d] = prior if lo == d else actual[lo:d].mean(axis=0)
    return f

load_f = forecast_avg(load, load_fj1)
pv_f   = forecast_avg(pv, pv_fj1)
net_actual = load - pv
net_f      = load_f - pv_f
```

```
err = net_actual[WINDOW:] - net_f[WINDOW:]
q80 = np.quantile(err, 0.80, axis=0)


def q2_day(d, E_start):
    price_d = price4[d]
    load_hedged = load_f[d] + q80
    plan = C.solve_plan_carry(price_d, load_hedged, pv_f[d],
   E_start)
    covered = net_f[d] + q80 + plan["s"]
    e = np.maximum(net_actual[d] - covered, 0.0)
    cost_plan = float(np.sum(price_d * plan["b"]) * C.DT)
    cost_emer = float(np.sum(5.0 * price_d * e) * C.DT)
    return plan, e, cost_plan, cost_emer


def q3_day(d, E_start):
    price_d = price4[d]
    load_d = load[d]; pv_actual = pv[d]
    pv_f = [None] * 4
    pv_f[0] = C.forecast_to_10min(fc[d, 0, :], 0, 0.0)
    pv_f[1] = C.forecast_to_10min(fc[d, 1, :], 6, pv_f[0][36])
    pv_f[2] = C.forecast_to_10min(fc[d, 2, :], 12, pv_f[1][72])
    pv_f[3] = C.forecast_to_10min(fc[d, 3, :], 18, pv_f[2][108])

    b_final = np.zeros(C.N); ch_final = np.zeros(C.N)
    dis_final = np.zeros(C.N); E_final = np.zeros(C.N)
    E_cur = E_start
    b0 = ch0 = dis0 = None
    for k in range(4):
        t0 = k * 36
        res = C.solve_plan_carry(price_d[t0:], load_d[t0:], pv_f[k
   ][t0:], E_cur)
        b_final[t0:t0 + 36]   = res["b"][:36]
        ch_final[t0:t0 + 36]  = res["ch"][:36]
        dis_final[t0:t0 + 36] = res["dis"][:36]
        E_final[t0:t0 + 36]   = res["E"][:36]
        E_cur = res["E"][35]
        if k == 0:
            b0, ch0, dis0 = res["b"], res["ch"], res["dis"]
    normal  = np.minimum(b0, b_final)
```

```
    deficit = np.maximum(b0 - b_final, 0.0)
    excess  = np.maximum(b_final - b0, 0.0)
    cost_plan = float(np.sum(normal * price_d) * C.DT)
    cost_adj  = float(np.sum((deficit * 0.5 + excess * 1.5) * price_d) * C.DT)
    covered = b_final + dis_final - ch_final
    e = np.maximum((load_d - pv_actual) - covered, 0.0)
    cost_emer = float(np.sum(5.0 * price_d * e) * C.DT)
    return b0, b_final, ch_final, dis_final, E_final, e, cost_plan, cost_adj, cost_emer


# 跨日连续：1月1日 0:00=6000，逐日顺延（不再日末归位）
E0_warm = C.warmup_storage(lambda d: price4[d], load, pv, START)
E_q2 = E0_warm
E_q3 = E0_warm

q2_plan, q2_emer, q2_chg = {}, {}, {}
q2_cp, q2_ce = [], []
q3_plan, q3_adj, q3_chg, q3_emer = {}, {}, {}, {}
q3_cp, q3_ca, q3_ce = [], [], []
for d in range(START, END):
    plan, e, cp, ce = q2_day(d, E_q2)
    E0_q2 = E_q2
    E_q2 = float(plan["E"][-1])
    q2_plan[dates[d]] = {"b": plan["b"], "price": price4[d]}
    q2_emer[dates[d]] = e
    q2_chg[dates[d]] = {"ch": plan["ch"], "dis": plan["dis"], "E_start": E0_q2, "E_end": plan["E"][-1]}
    q2_cp.append(cp); q2_ce.append(ce)
    b0, b_adj, ch_f, dis_f, E_f, e3, cp3, ca3, ce3 = q3_day(d, E_q3)
    E0_q3 = E_q3
    E_q3 = float(E_f[-1])
    q3_plan[dates[d]] = b0
    q3_adj[dates[d]] = b_adj
    q3_chg[dates[d]] = {"ch": ch_f, "dis": dis_f, "E_start": E0_q3, "E_end": E_f[-1]}
    q3_emer[dates[d]] = e3
    q3_cp.append(cp3); q3_ca.append(ca3); q3_ce.append(ce3)

```

```
q2_cp = np.array(q2_cp); q2_ce = np.array(q2_ce)
q3_cp = np.array(q3_cp); q3_ca = np.array(q3_ca); q3_ce = np.array(q3_ce)
q2_total = q2_cp + q2_ce
q3_total = q3_cp + q3_ca + q3_ce

print("问题4 波动电价结果（2025.2.1 ~ 12.31，共", END - START, "天）")
print(f"[问题2 波动电价] 计划购电费 : {q2_cp.sum():,.2f} 元")
print(f"[问题2 波动电价] 紧急购电费 : {q2_ce.sum():,.2f} 元")
print(f"[问题2 波动电价] 总购电费   : {q2_total.sum():,.2f} 元 "
      f"(紧急占比 {q2_ce.sum()/q2_total.sum()*100:.2f}%)")
print(f"[问题3 波动电价] 计划购电费 : {q3_cp.sum():,.2f} 元")
print(f"[问题3 波动电价] 调整相关费 : {q3_ca.sum():,.2f} 元")
print(f"[问题3 波动电价] 紧急购电费 : {q3_ce.sum():,.2f} 元")
print(f"[问题3 波动电价] 总购电费   : {q3_total.sum():,.2f} 元 "
      f"(紧急占比 {q3_ce.sum()/q3_total.sum()*100:.2f}%)")

out2 = os.path.join(HERE, "result4-2.xlsx")
if os.path.exists(out2): os.remove(out2)
C.write_plan_sheet(q2_plan, out2, "计划购电量")
C.write_charge_sheet(q2_chg, out2, "充放电量")
C.write_emergency_sheet(q2_emer, out2, "紧急购电量")
print(f"已保存 result4-2.xlsx 到 {HERE}")

out3 = os.path.join(HERE, "result4-3.xlsx")
if os.path.exists(out3): os.remove(out3)

def write_purchase_sheet(rec, sheet, fee_map):
    cols = ["日期\\时间"] + C.period_labels_all() + ["全天购电量", "全天购电费"]
    rows = []
    for d in range(START, END):
        dt = dates[d]
        kwh = np.round(rec[dt] * C.DT, 4)
        rows.append([C._fmt_date(dt)] + list(kwh) + [round(float(np.sum(kwh)), 4),
                                                     round(float(fee_map[dt]), 2)])
    C._append_sheet(pd.DataFrame(rows, columns=cols), out3, sheet)

fee_plan = {dates[d]: q3_cp[d - START] for d in range(START, END)}
```

```
fee_adj = {dates[d]: q3_ca[d - START] for d in range(START, END)}
write_purchase_sheet(q3_plan, "计划购电量", fee_plan)
write_purchase_sheet(q3_adj, "调整购电量", fee_adj)
chg_rows = []
for d in range(START, END):
    rec = q3_chg[dates[d]]
    ch4, dis4 = C.aggregate_4h(rec["ch"]), C.aggregate_4h(rec["dis"])
    date_str = C._fmt_date(dates[d])
    for p in range(6):
        tc = "0:00" if p == 0 else ("24:00" if p == 1 else "")
        ec = round(rec["E_start"], 1) if p == 0 else (round(rec["E_end"], 1) if p == 1 else "")
        chg_rows.append([date_str if p == 0 else "", C.FOUR_H_LABELS[p],
                         round(ch4[p], 2), round(dis4[p], 2), tc, ec])
C._append_sheet(pd.DataFrame(chg_rows, columns=["日期", "时间段", "充电量", "放电量", "时刻", "储电量"]),
                out3, "充放电量")
emer_rows = []
for d in range(START, END):
    e = q3_emer[dates[d]]
    date_str = C._fmt_date(dates[d])
    inter = C.merge_contiguous(e)
    if not inter:
        emer_rows.append([date_str, "", ""])
    else:
        for (s, t, kwh) in inter:
            lbl = f"{C.period_label(s).split('-')[0]}-{C.period_label(t-1).split('-')[1]}"
            emer_rows.append([date_str, lbl, round(kwh, 4)])
C._append_sheet(pd.DataFrame(emer_rows, columns=["日期", "购电时间段", "购电量"]),
                out3, "紧急购电量")
print(f"已保存 result4-3.xlsx 到 {HERE}")

summary = pd.DataFrame({
    "日期": [str(pd.Timestamp(dates[d]).date()) for d in range(START, END)],
    "问题2计划费": q2_cp, "问题2紧急费": q2_ce, "问题2总费": q2_total,
```

```
    "问题3计划费": q3_cp, "问题3调整费": q3_ca, "问题3紧急费": q3_ce, "问题3总费": q3_total,
})
C.write_summary(summary, os.path.join(HERE, "_q4_summary.csv"))
print("已保存汇总 _q4_summary.csv")

np.savez(os.path.join(HERE, "_q4_data.npz"),
         dates=dates[START:END], price4=price4[START:END],
         q2_cost=q2_total, q2_cp=q2_cp, q2_ce=q2_ce,
         q3_cost=q3_total, q3_cp=q3_cp, q3_ca=q3_ca, q3_ce=q3_ce)
print("已保存绘图数据 _q4_data.npz")
```

Listing 11　solve_q4.py（问题 4 波动电价下重解问题 2/3）

B.5.2　solve ——问题 4 电价预测

```
# -*- coding: utf-8 -*-
"""
问题4 补充：电价未知（需预测）时的策略求解与对比

假设附件4 的波动电价在制定购电计划时未知，只能用历史电价做“前5日移动平均”预测，
计划/调度依据预测电价制定，但结算仍按真实电价计价。与“电价已知”的基准结果对比，
量化电价预测误差带来的成本增量，并生成对比图。

思路（与负荷/光伏预报同源的 5 日移动平均）：
    price_f[d] = mean(price4[max(0,d-5):d])     （首 5 日用附件1 固定电价作先验）
"""
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()
P = C.PALETTE
```

```
25
26 price_fix, load_fj1, pv_fj1 = C.load_fj1()
27 dates, load, pv = C.load_fj2()
28 _, fc = C.load_fj3()
29 _, price4 = C.load_fj4()
30
31 n_days = load.shape[0]
32 START, END = 31, n_days
33 WINDOW = 5
34
35
36 def forecast_avg(actual, prior):
37     n = actual.shape[0]
38     f = np.empty_like(actual)
39     for d in range(n):
40         lo = max(0, d - WINDOW)
41         f[d] = prior if lo == d else actual[lo:d].mean(axis=0)
42     return f
43
44
45 # ---- 电价预测：前 5 日移动平均（首日用附件1 固定电价作先验） ----
46 price_f = forecast_avg(price4, price_fix)
47
48 load_f = forecast_avg(load, load_fj1)
49 pv_f = forecast_avg(pv, pv_fj1)
50 net_actual = load - pv
51 net_f = load_f - pv_f
52 err = net_actual[WINDOW:] - net_f[WINDOW:]
53 q80 = np.quantile(err, 0.80, axis=0)
54
55 # 电价预测误差（分析窗口内，相对真实电价）
56 price_err = price4[START:END] - price_f[START:END]
57 mape = float(np.mean(np.abs(price_err) / (np.abs(price4[START:END])
       + 1e-9)) * 100)
58 rmse = float(np.sqrt(np.mean(price_err ** 2)))
59
60
61 def q2_day_fc(d, E_start):
62     """两阶段：按预测电价做计划，按真实电价结算。"""
63     price_actual = price4[d]
64     price_fc_d = price_f[d]
```

```
    load_hedged = load_f[d] + q80
    plan = C.solve_plan_carry(price_fc_d, load_hedged, pv_f[d],
   E_start)
    covered = net_f[d] + q80 + plan["s"]
    e = np.maximum(net_actual[d] - covered, 0.0)
    cost_plan = float(np.sum(price_actual * plan["b"]) * C.DT)
    cost_emer = float(np.sum(5.0 * price_actual * e) * C.DT)
    return plan, e, cost_plan, cost_emer


def q3_day_fc(d, E_start):
    """滚动 MPC：每次重规划按预测电价，结算按真实电价。"""
    price_actual = price4[d]
    price_fc_d = price_f[d]
    load_d = load[d]; pv_actual = pv[d]
    pv_f = [None] * 4
    pv_f[0] = C.forecast_to_10min(fc[d, 0, :], 0, 0.0)
    pv_f[1] = C.forecast_to_10min(fc[d, 1, :], 6, pv_f[0][36])
    pv_f[2] = C.forecast_to_10min(fc[d, 2, :], 12, pv_f[1][72])
    pv_f[3] = C.forecast_to_10min(fc[d, 3, :], 18, pv_f[2][108])

    b_final = np.zeros(C.N); ch_final = np.zeros(C.N)
    dis_final = np.zeros(C.N); E_final = np.zeros(C.N)
    E_cur = E_start
    b0 = ch0 = dis0 = None
    for k in range(4):
        t0 = k * 36
        res = C.solve_plan_carry(price_fc_d[t0:], load_d[t0:], pv_f
   [k][t0:], E_cur)
        b_final[t0:t0 + 36] = res["b"][:36]
        ch_final[t0:t0 + 36] = res["ch"][:36]
        dis_final[t0:t0 + 36] = res["dis"][:36]
        E_final[t0:t0 + 36] = res["E"][:36]
        E_cur = res["E"][35]
        if k == 0:
            b0, ch0, dis0 = res["b"], res["ch"], res["dis"]
    normal = np.minimum(b0, b_final)
    deficit = np.maximum(b0 - b_final, 0.0)
    excess = np.maximum(b_final - b0, 0.0)
    cost_plan = float(np.sum(normal * price_actual) * C.DT)
    cost_adj = float(np.sum((deficit * 0.5 + excess * 1.5) *
```

```
    price_actual) * C.DT)
    covered = b_final + dis_final - ch_final
    e = np.maximum((load_d - pv_actual) - covered, 0.0)
    cost_emer = float(np.sum(5.0 * price_actual * e) * C.DT)
    return cost_plan, cost_adj, cost_emer, float(E_final[-1])


# ---- 预热：已知场景用真实电价，预测场景用预测电价，均从 1月1日 0:00=6000 连续演化 ----
E0_known = C.warmup_storage(lambda d: price4[d], load, pv, START)
E0_fc = C.warmup_storage(lambda d: price_f[d], load, pv, START)

q2_cp, q2_ce = [], []
q3_cp, q3_ca, q3_ce = [], [], []
E_q2, E_q3 = E0_fc, E0_fc
for d in range(START, END):
    plan, e, cp, ce = q2_day_fc(d, E_q2)
    E_q2 = float(plan["E"][-1])
    q2_cp.append(cp); q2_ce.append(ce)
    cp3, ca3, ce3, E_end3 = q3_day_fc(d, E_q3)
    E_q3 = E_end3
    q3_cp.append(cp3); q3_ca.append(ca3); q3_ce.append(ce3)

q2_cp = np.array(q2_cp); q2_ce = np.array(q2_ce)
q3_cp = np.array(q3_cp); q3_ca = np.array(q3_ca); q3_ce = np.array(q3_ce)
q2_fc = q2_cp + q2_ce
q3_fc = q3_cp + q3_ca + q3_ce

# ---- 已知电价基准（直接读 solve_q4 的连续储能结果） ----
base = np.load(os.path.join(HERE, "_q4_data.npz"))
q2_known = base["q2_cost"].sum()
q3_known = base["q3_cost"].sum()

print("问题4 电价预测 vs 已知 对比（2025.2.1 ~ 12.31，共", END - START, "天）")
print(f"电价预测方法：前 5 日移动平均（首日附件1 固定价先验）")
print(f"电价预测误差：MAPE {mape:.2f}%，RMSE {rmse:.3f} 元/kWh")
print()
print(f"[问题2 两阶段] 已知电价总费 : {q2_known:,.2f} 元")
print(f"[问题2 两阶段] 预测电价总费 : {q2_fc.sum():,.2f} 元  "
```

```
      f"(增量 {(q2_fc.sum()-q2_known):,.2f} 元, {(q2_fc.sum()-q2_known)/q2_known*100:.2f}%)")
print(f"[问题3 滚动]   已知电价总费 : {q3_known:,.2f} 元")
print(f"[问题3 滚动]   预测电价总费 : {q3_fc.sum():,.2f} 元  "
      f"(增量 {(q3_fc.sum()-q3_known):,.2f} 元, {(q3_fc.sum()-q3_known)/q3_known*100:.2f}%)")

summary = pd.DataFrame({
    "日期": [str(pd.Timestamp(dates[d]).date()) for d in range(START, END)],
    "问题2预测总费": q2_fc, "问题3预测总费": q3_fc,
})
C.write_summary(summary, os.path.join(HERE, "_q4_price_forecast_summary.csv"))

# ---- 图19: 电价已知 vs 预测 总费用对比 ----
labels = ["问题2 · 两阶段\n电价已知", "问题2 · 两阶段\n电价预测",
          "问题3 · 滚动\n电价已知", "问题3 · 滚动\n电价预测"]
vals = [q2_known, q2_fc.sum(), q3_known, q3_fc.sum()]
colors = [P["blue"], P["red"], P["green"], P["orange"]]
fig, ax = plt.subplots(figsize=(9, 5.2))
bars = ax.bar(labels, vals, color=colors, alpha=0.92, width=0.62)
for b_bar, v in zip(bars, vals):
    ax.text(b_bar.get_x() + b_bar.get_width() / 2, v + max(vals) * 0.012,
            f"{v/1e4:,.1f} 万元", ha="center", fontsize=11, weight="bold")
# 标注预测相对已知的增量
for i_known, i_fc in [(0, 1), (2, 3)]:
    inc = vals[i_fc] - vals[i_known]
    ax.annotate(f"+{inc/1e4:.1f} 万元",
                xy=(i_fc, vals[i_fc] * 0.5),
                xytext=(i_fc, vals[i_fc] * 0.5 + max(vals) * 0.10),
                fontsize=10, color="#FFFFFF", weight="bold", ha="center")
ax.set_ylabel("全年总购电费 (元)")
ax.set_title("图19  问题4 电价「未知→预测」与「已知」的总费用对比")
ax.grid(axis="x", visible=False)
fig.tight_layout()
fig.savefig(os.path.join(OUT, "图19_电价已知vs预测对比.png"))
plt.close(fig)
```

```

# ---- 图20：典型日电价预测 vs 真实 + 误差 ----
di = 150
t = np.arange(C.N) / 6.0
fig, axs = plt.subplots(2, 1, figsize=(9.5, 6.4), sharex=True,
                        gridspec_kw={"height_ratios": [2, 1]})
ax = axs[0]
ax.plot(t, price4[di], color=P["dark"], lw=2.0, label="真实波动电价（附件4）")
ax.plot(t, price_f[di], color=P["red"], lw=1.5, ls="--", label="预测电价（前5日移动平均）")
ax.set_ylabel("电价（元/kWh)"); ax.set_xlim(0, 24)
ax.set_title(f"  2025-{pd.Timestamp(dates[di]).date()}  电价预测 vs 真实", fontsize=10.5, loc="left")
ax.legend(loc="upper left", frameon=True, facecolor="white", edgecolor="#E4E7EB")
ax.margins(x=0)
ax2 = axs[1]
ax2.fill_between(t, price4[di] - price_f[di], 0, color=P["red"], alpha=0.25)
ax2.plot(t, price4[di] - price_f[di], color=P["red"], lw=1.2)
ax2.axhline(0, color="#C7CCD4", lw=0.8)
ax2.set_xlabel("时间（h)"); ax2.set_ylabel("预测误差（元/kWh)")
ax2.set_title("  预测误差（真实 - 预测）", fontsize=10.5, loc="left")
ax2.margins(x=0)
fig.suptitle(f"图20  波动电价预测示意（全年 MAPE {mape:.1f}%）", fontsize=13, weight="bold", y=0.99)
fig.tight_layout(rect=[0, 0, 1, 0.96])
fig.savefig(os.path.join(OUT, "图20_电价预测误差.png"))
plt.close(fig)

print("\n已保存 _q4_price_forecast_summary.csv 与 图19/图20")
```

Listing 12　solve_q4_price_forecast.py（问题 4 电价预测）

B.5.3　plot ——问题 4 绘图

```
# -*- coding: utf-8 -*-
import os
import sys
import numpy as np
import pandas as pd
```

```

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import common as C

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "图")
os.makedirs(OUT, exist_ok=True)
plt = C.setup_plot()

d = np.load(os.path.join(HERE, "_q4_data.npz"))
dates = d["dates"]; price4 = d["price4"]
q2_cost = d["q2_cost"]; q3_cost = d["q3_cost"]

n_days = len(dates)
day_idx = np.arange(n_days)
date_lbl = [str(pd.Timestamp(dates[i]).date())[5:] for i in range(n_days)]
t = np.arange(C.N) / 6.0
P = C.PALETTE


def fig15_price_compare():
    price_fix, _, _ = C.load_fj1()
    di = 150
    fig, ax = plt.subplots(figsize=(9.5, 4.4))
    ax.plot(t, price_fix, color=P["blue"], lw=2.0, label="附件1 固定电价")
    ax.plot(t, price4[di], color=P["red"], lw=1.5, label=f"{date_lbl[di]} 波动电价")
    ax.set_xlabel("时间 (h)"); ax.set_ylabel("电价 (元/kWh)")
    ax.set_title("图15　波动电价与固定电价对比（典型日）")
    ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
    ax.margins(x=0)
    ax.legend(loc="upper left")
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "图15_波动电价对比.png"))
    plt.close(fig)


def fig16_heatmap():
```

```
    fig, ax = plt.subplots(figsize=(10, 7))
    im = ax.imshow(price4.T, aspect="auto", origin="lower", cmap="viridis",
                   extent=[0, n_days, 0, 24])
    ax.grid(False)
    ax.set_xlabel("日期"); ax.set_ylabel("时间 (h)")
    ax.set_title("图16  全年波动电价热力图（颜色 = 电价 元/kWh）", fontsize=12, weight="bold")
    ax.set_xticks(np.arange(0, n_days, 30))
    ax.set_xticklabels([date_lbl[i] for i in range(0, n_days, 30)], fontsize=8)
    ax.set_yticks(np.arange(0, 25, 3))
    cb = fig.colorbar(im, ax=ax, pad=0.015, fraction=0.046)
    cb.set_label("电价 (元/kWh)")
    cb.outline.set_visible(False)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "图16_电价热力图.png"))
    plt.close(fig)


def fig17_linkage():
    _, load, pv = C.load_fj2()
    di = 150
    di_full = 31 + di
    price_d = price4[di]
    res = C.solve_plan(price_d, load[di_full], pv[di_full])
    ch, dis, E = res["ch"], res["dis"], res["E"]
    Efull = np.concatenate([[C.E0], E])
    fig, ax = plt.subplots(figsize=(9.5, 4.8))
    ax.plot(t, price_d, color=P["red"], lw=2.0, label="波动电价 (左轴)")
    ax.set_xlabel("时间 (h)"); ax.set_ylabel("电价 (元/kWh)", color=P["red"])
    ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 3))
    ax.margins(x=0)
    ax2 = ax.twinx()
    ax2.spines["right"].set_visible(True); ax2.spines["right"].set_color("#C7CCD4")
    ax2.plot(np.arange(C.N + 1) / 6.0, Efull, color=P["purple"], lw=2.0, label="储电量 (右轴)")
    ax2.fill_between(t, 0, ch, color=P["purple"], alpha=0.20, label
```

```
    ="充电")
77   ax2.fill_between(t, 0, -dis, color=P["green"], alpha=0.20,
    label="放电")
78   ax2.set_ylabel("储电量 (kWh) / 充放电功率 (kW)", color=P["purple"
    ])
79   ax2.axhline(0, color="#C7CCD4", lw=0.7)
80   h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.
    get_legend_handles_labels()
81   ax.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=8.5,
82             frameon=True, facecolor="white", edgecolor="#E4E7EB")
83   ax.set_title(f"图17　波动电价下储能低充高放（{date_lbl[di]}）")
84   fig.tight_layout()
85   fig.savefig(os.path.join(OUT, "图17_电价储能联动.png"))
86   plt.close(fig)
87
88
89 def fig18_compare():
90     def csv_sum(path, col):
91         try:
92             df = pd.read_csv(path)
93             return float(df[col].sum())
94         except Exception:
95             return 0.0
96
97     p2 = csv_sum(os.path.join(HERE, "..", "问题2", "_q2_summary.csv"
    ), "总购电费(元)")
98     p3 = csv_sum(os.path.join(HERE, "..", "问题3", "_q3_summary.csv"
    ), "总购电费(元)")
99     p42 = q2_cost.sum()
100    p43 = q3_cost.sum()
101
102    labels = ["问题2\n固定价 · 两阶段", "问题3\n固定价 · 滚动", "问题4-2\n波
    动价 · 两阶段", "问题4-3\n波动价 · 滚动"]
103    vals = [p2, p3, p42, p43]
104    colors = [P["blue"], P["green"], P["red"], P["orange"]]
105    fig, ax = plt.subplots(figsize=(8.5, 5))
106    bars = ax.bar(labels, vals, color=colors, alpha=0.9, width=0.6)
107    for b_bar, v in zip(bars, vals):
108        ax.text(b_bar.get_x() + b_bar.get_width() / 2, v + max(vals
    ) * 0.015,
109                f"{v/1e4:,.0f} 万元", ha="center", fontsize=10.5,
```

```
    weight="bold")
    ax.set_ylabel("全年总购电费（元）")
    ax.set_title("图18　四种策略全年总购电费对比")
    ax.grid(axis="x", visible=False)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "图18_四问总费用对比.png"))
    plt.close(fig)


if __name__ == "__main__":
    fig15_price_compare(); fig16_heatmap(); fig17_linkage();
    fig18_compare()
    print("问题4 图表已生成到", OUT)
```

Listing 13　plot_q4.py（问题 4 图表生成）